

Sisteme și algoritmi distribuiți

Curs 3

Cuprins

- Algoritmi sincroni
- Alegere lider pe inel (sincron)
- Alegere lider pe topologii generale (sincron)

Algoritmi sincroni în SD

SPMD

Un algoritm distribuit sincron reprezintă un set de operații de calcul/comunicație *executat în iterații/runde* (contorizate de t) pe nodurile sistemului, cu scopul rezolvării unei sarcini concrete. *stare optimă*

- Denumim starea locală a nodului i la momentul t cu $x_i(t)$. Scopul algoritmului este conducerea lui $x_i(t)$ către starea optimă $x_i(\infty)$
- La următorul moment de timp starea x_i suferă o transformare bazată pe pașii algoritmului și informația provenită de la vecini. Pe scurt,

$$x_i(t + 1) := f_i(x(t))$$

unde $f_i(\cdot)$ reprezintă funcția de transformare asociată nodului i . Funcția $f_i(\cdot)$ definește însuși algoritmul SPMD.

Algoritmi sincroni în SD

La următorul moment de timp starea x_i suferă o transformare bazată pe pașii algoritmului și informația provenită de la vecini. Pe scurt,

$$x_i(t+1) := f_i(x(t)) := \frac{x_i(t) + x_j(t)}{2} \rightarrow P_j \text{ send } (x_j(t+1))$$

$P_i \text{ recv } (x_j(t+1))$

unde $f_i(\cdot)$ reprezintă funcția de transformare asociată nodului i .

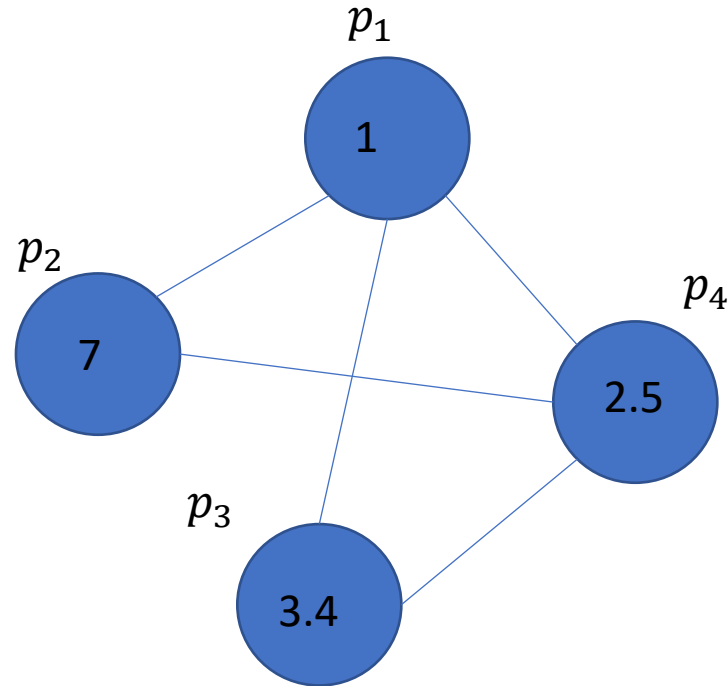
Funcția $f_i(\cdot)$ este definită în format SPMD și este compusă din una sau mai multe operații (e.g. aritmetice, numerice) asupra variabilelor locale din memoria nodului i .

Medie şir de numere reale

Problemă [Distributed averaging]: Vectorul $x(0)$ este distribuit peste n noduri, astfel încât $x_i(0)$ se află în memoria locală a nodului i .

Calculează distribuit media aritmetică $m = \frac{\sum_i x_i(0)}{n}$ a vectorului $x(0)$, încât la final $x_i(\infty) = m$.

$$x(0) = \begin{bmatrix} 1 \\ 7 \\ 3.4 \\ 2.5 \end{bmatrix}$$



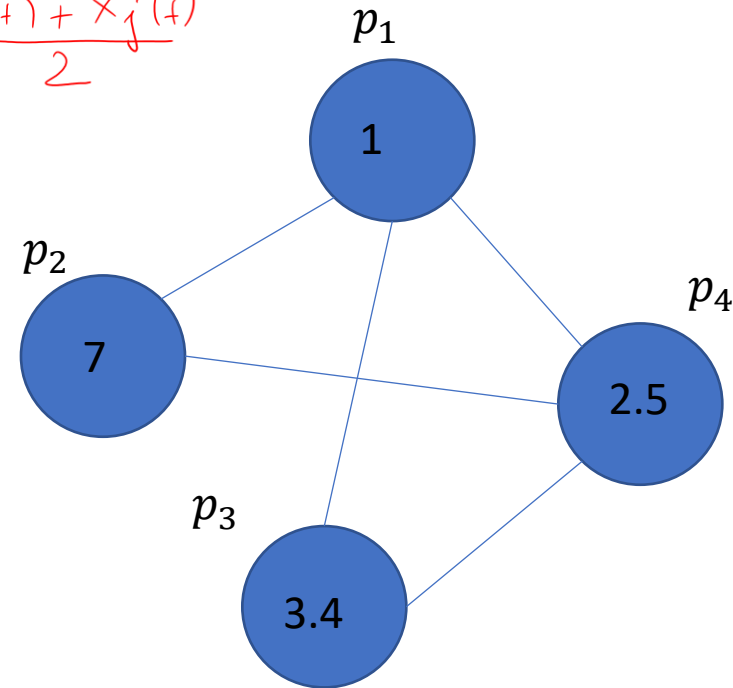
Medie şir de numere reale

Problemă [Distributed averaging]: Vectorul $x(0)$ este distribuit peste n noduri, astfel încât $x_i(0)$ se află în memoria locală a nodului i . Calculează distribuit media aritmetică $m = \sum_i x_i(0)/n$ a vectorului $x(0)$, încât la final $x_i(\infty) = m$.

Algorithm: Alege $j \in N_i$ şi actualizează $x_i(t+1) = f_i(x(t)) = \frac{x_i(t) + x_j(t)}{2}$

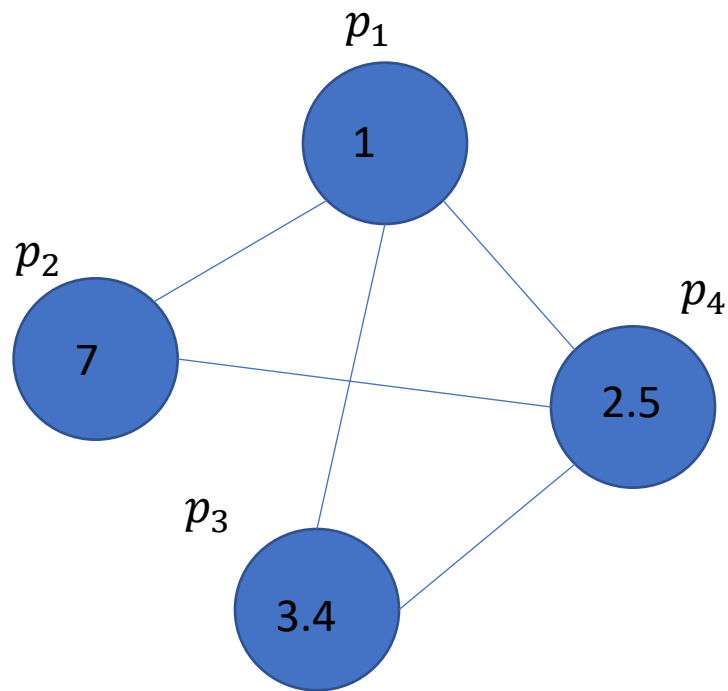
Handwritten notes:
 $x_j(t+1) = \frac{x_i(t) + x_j(t)}{2}$

$$x(0) = \begin{bmatrix} 1 \\ 7 \\ 3.4 \\ 2.5 \end{bmatrix}$$



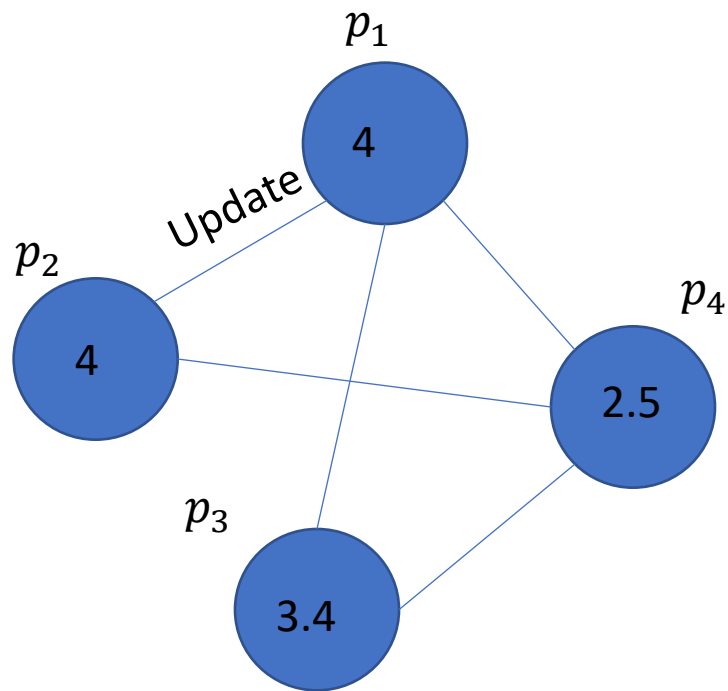
Medie șir de numere reale

$$\underbrace{x(0)} = \begin{bmatrix} 1 \\ 7 \\ 3.4 \\ 2.5 \end{bmatrix} \in \mathbb{R}^4$$



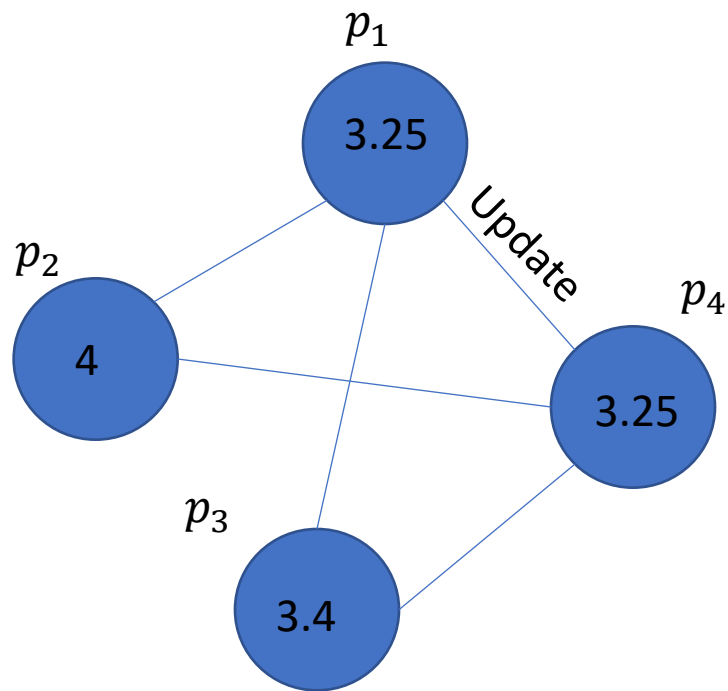
Medie șir de numere reale

$$x(1) = \begin{bmatrix} 4 \\ 4 \\ 3.4 \\ 2.5 \end{bmatrix}$$



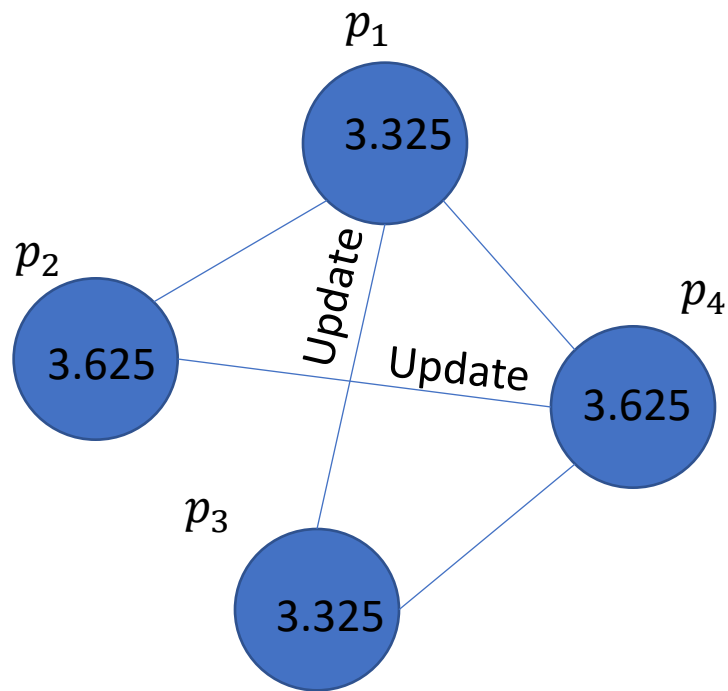
Medie șir de numere reale

$$x(2) = \begin{bmatrix} 3.25 \\ 4 \\ 3.4 \\ 3.25 \end{bmatrix}$$



Medie șir de numere reale

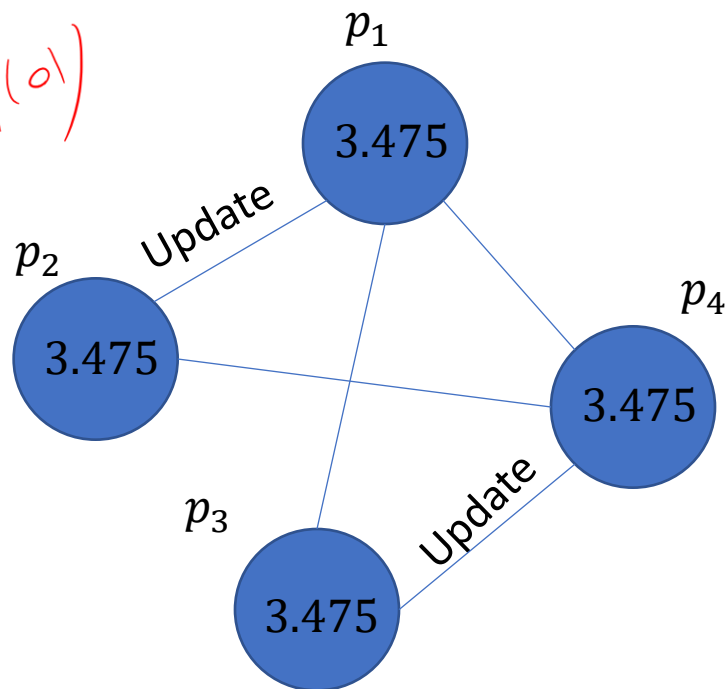
$$x(4) = \begin{bmatrix} 3.325 \\ 3.625 \\ 3.325 \\ 3.625 \end{bmatrix}$$



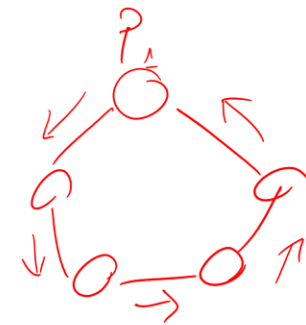
Medie șir de numere reale

$$\underbrace{x(6)} = \begin{bmatrix} 3.475 \\ 3.475 \\ 3.475 \\ 3.475 \end{bmatrix} = x^*$$

$m = \frac{1}{4}(x_1(0) + \dots + x_4(0))$



Algoritmi Sincroni în SD



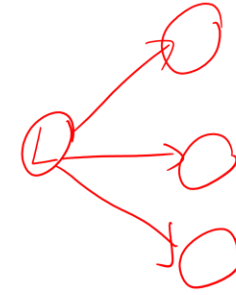
La finalul iterației t fiecare nod finalizează calculul local, și comunicația cu vecinii, specifice iterației t

$$\underbrace{x(t+1)} := \begin{bmatrix} x_1(t+1) \\ \dots \\ x_n(t+1) \end{bmatrix} = \begin{bmatrix} f_1(x(t)) \\ \dots \\ f_n(x(t)) \end{bmatrix} = \underbrace{F(x(t))}_{t > T \text{ atunci STOP}}$$

e.j.
 $F: \mathbb{R}^n \rightarrow \mathbb{R}^n$

- Există o margine superioară pe timpul de comunicație între oricare două noduri.
- Adesea implementarea unui criteriu de oprire este o operație dificilă!

Alegere Lider (Leader Election)



În multe aplicații este necesară alegerea unui nod pentru operații particulare (e.g. difuzare, distribuție, master-slave).

Fiecare nod are un ^{token} ID unic, ales dintr-un spațiu total ordonat. $P_1, P_2, \dots, P_m$

Convenție: Lider = nodul cu ID-ul maxim. $ID_{max} \geq ID_j, \forall j$

Algoritmii de LE realizează *de facto* calculul distribuit al funcției
 $\max\{id_1, id_2, \dots, id_n\}$

Alegere Lider (Leader Election)

Starea locală a nodului i specifică calitatea de lider/non-lider:

$$x_i(t) \in \{\text{lider}, \text{non-lider}\}$$

Funcția de transformare asociată nodului i :

$$f_i(\{x_j(t) | j \in N_i\})$$

decide dacă la iterația curentă nodul i devine sau nu lider.

Funcția f_i se reduce la una sau mai multe operații asupra memoriei locale M_i a nodului P_i .

Alegere Lider (Leader Election)

Starea locală a nodului i specifică calitatea de lider/non-lider:

$$x_i(t) \in \{lider, non - lider\}$$

Funcția de transformare asociată nodului i :

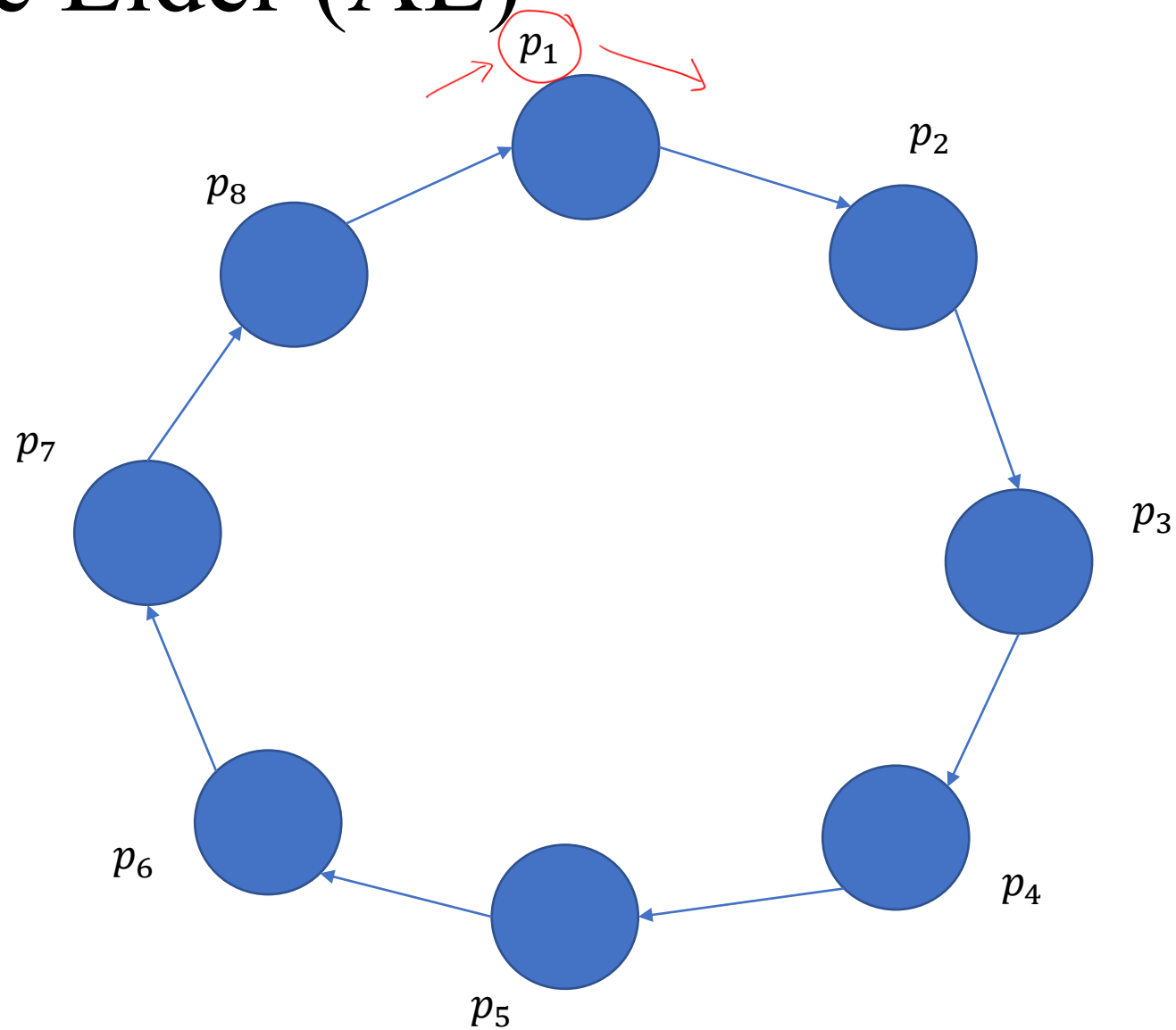
$$f_i(\{x_j(t) | j \in N_i\})$$

decide dacă la iterația curentă nodul i devine sau nu lider.

Asimptotic, soluția problemei este aducerea sistemului în starea:

$$x_i^* = \begin{cases} lider, & i = \operatorname{argmax}_j id_j \\ non - lider, & altfel \end{cases}$$

Alegere Lider (AL)

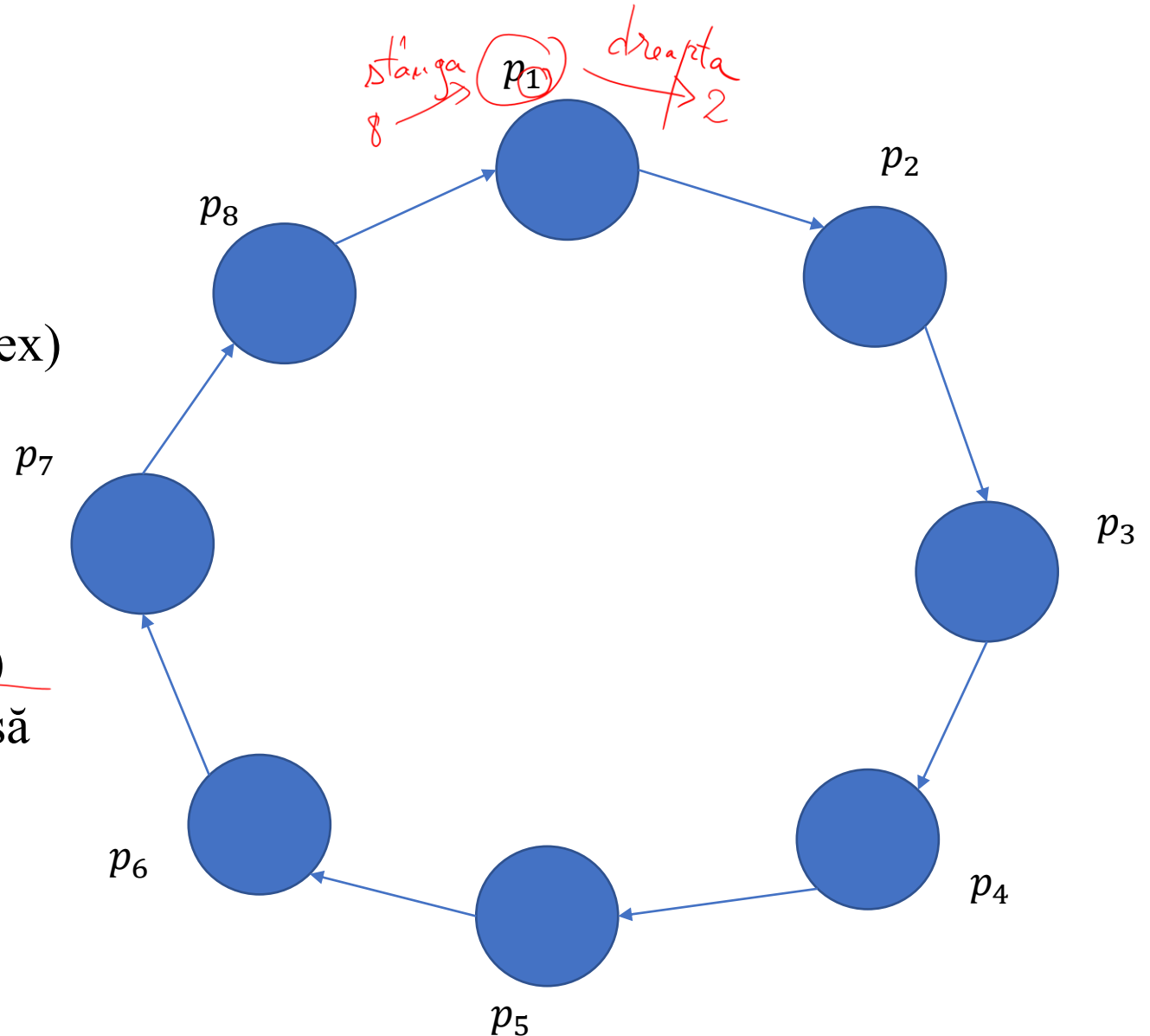


Alegere Lider (AL)

Ipoteze: P_i cunoaște

1. Topologie inel unidirecțional
2. Poziția proprie relativă în topologie (index)
3. Numărul de noduri n

Problema: Specifică un program SPMD (f_i) astfel încât, într-un număr minim de iterații să asigurăm convergența stării globale la starea optimă, *i.e.* $x_i(T) = x_i^*$.



Alegere Lider (coordonate globale)

Algoritm **AlegeLiderInel()**:

M_i : - int n (număr noduri)

- int i (index) *MPI_Rank()*

- int id (token)

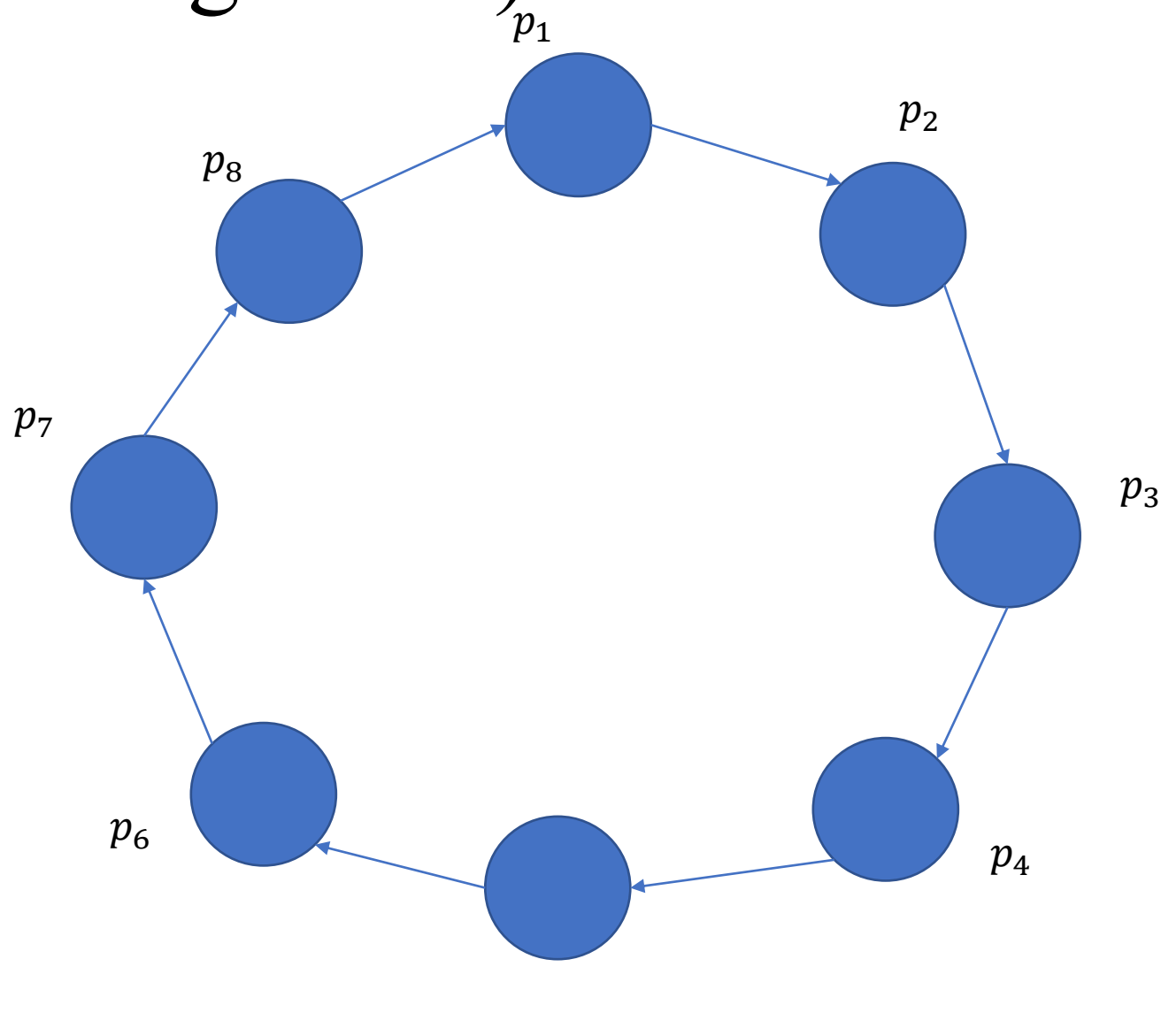
- int id_aux, id_max (memorează id maxim)

% Faza I: max ID

- Calculează $\max\{id_1, id_2, \dots, id_n\}$
- Rezultatul va fi stocat într-un nod particular

% Faza II: Difuzare Max ID (Broadcast)

- Rezultatul este difuzat peste tot inelul
- Stările x_i sunt ajustate conform rezultatului



Alegere Lider (coordonate globale)

M_i : - int n (număr noduri)

- int i (index)

- int id (token)

- int id_aux, id_max (memorează id maxim)

% Faza max ID

Funcție transformare nod i $f_i()$:

1. **If (i == 1):**

1. send(id, 2);

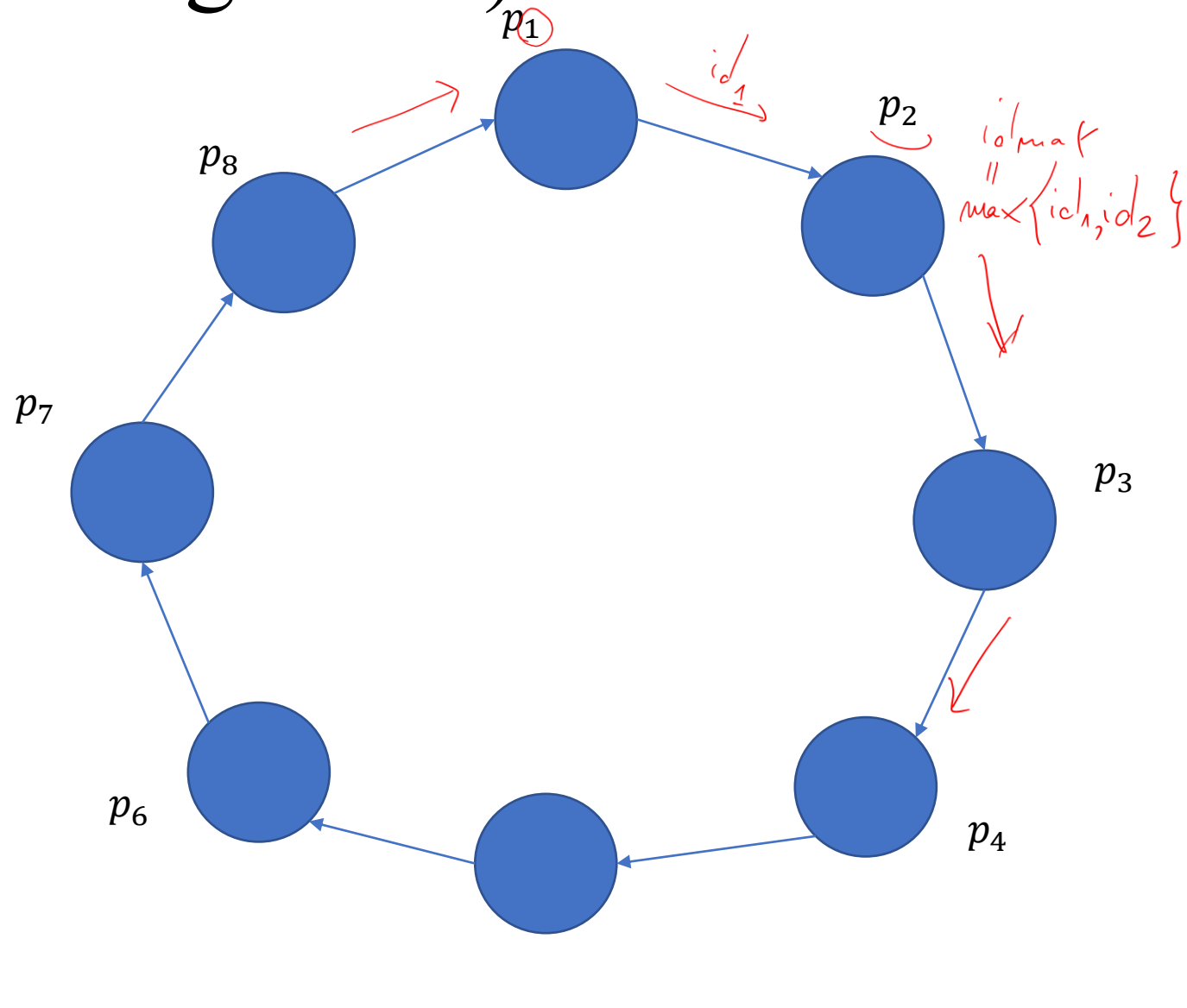
2. id_aux = recv(n);

2. **else:**

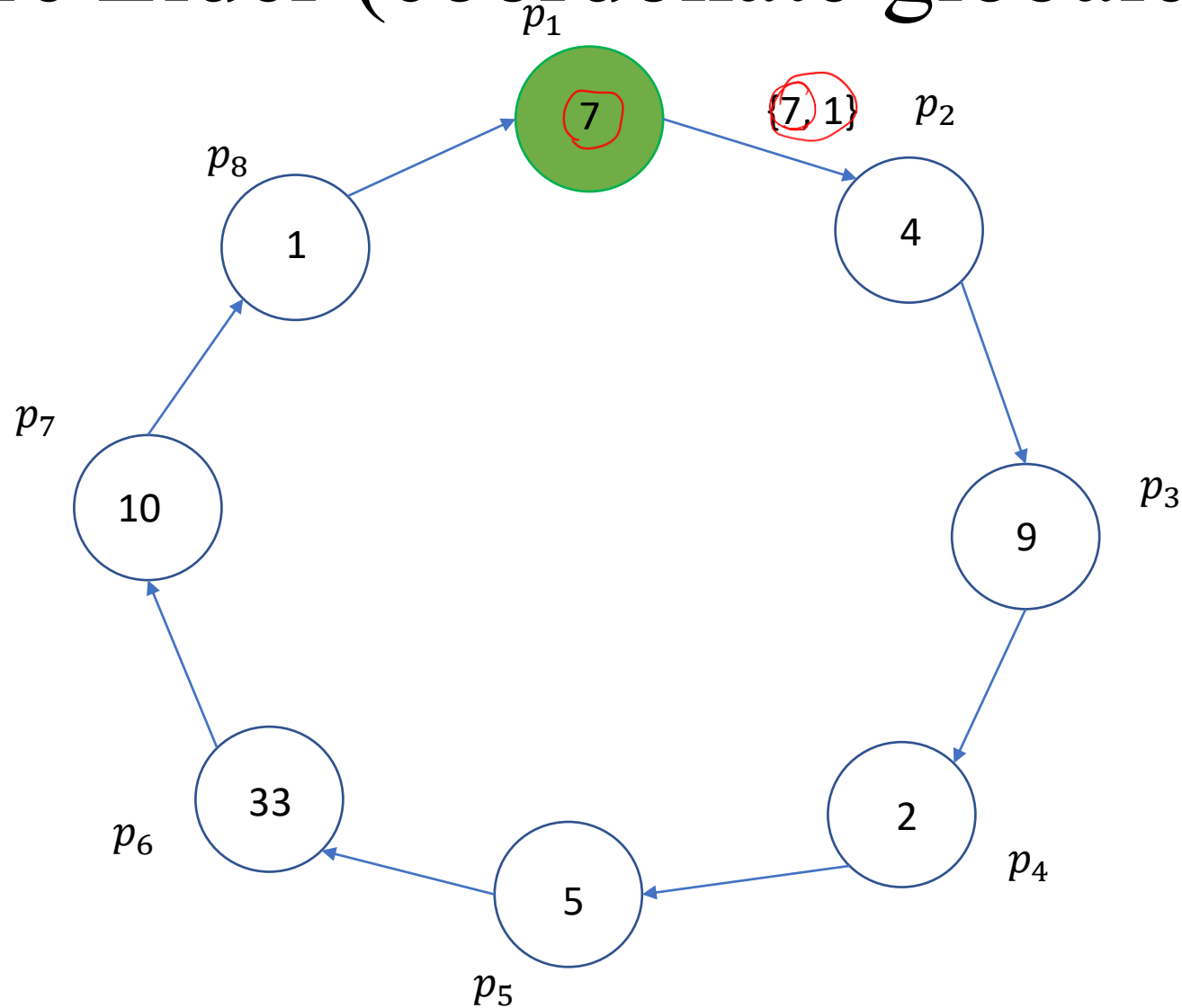
1. id_aux = recv(i - 1 mod n);

2. idmax = max(id, id_aux);

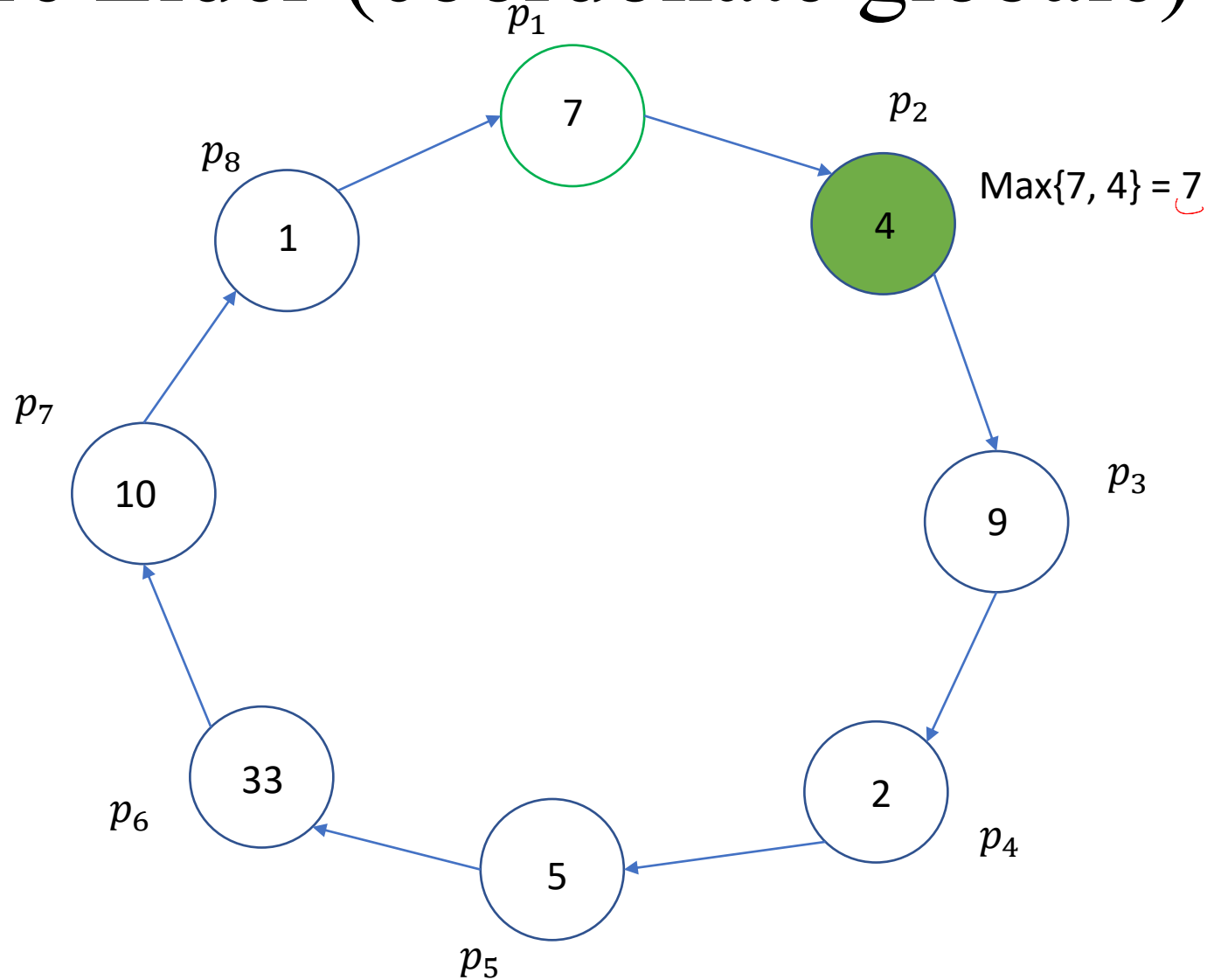
3. send(idmax, i + 1 mod n);



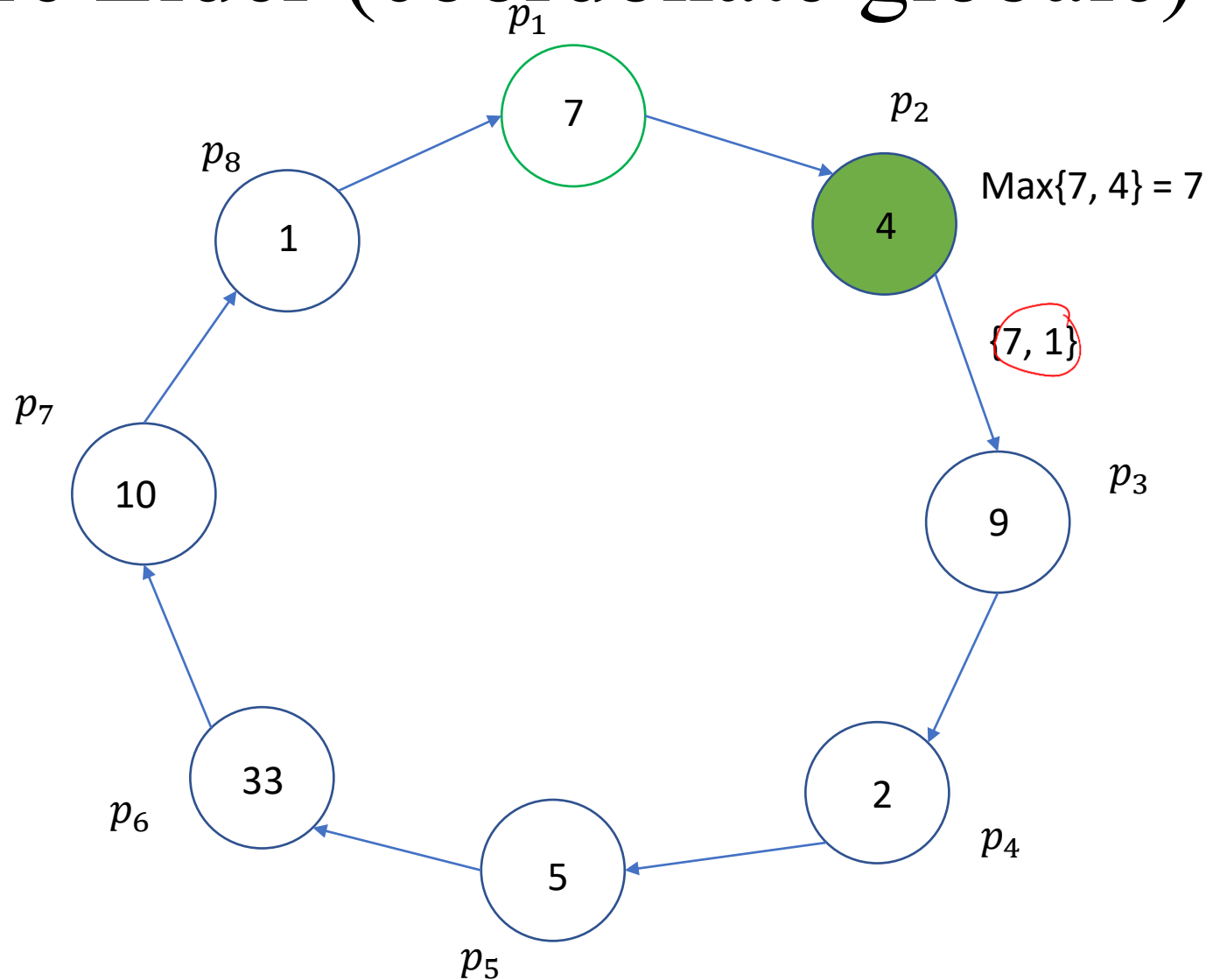
Alegere Lider (coordonate globale)



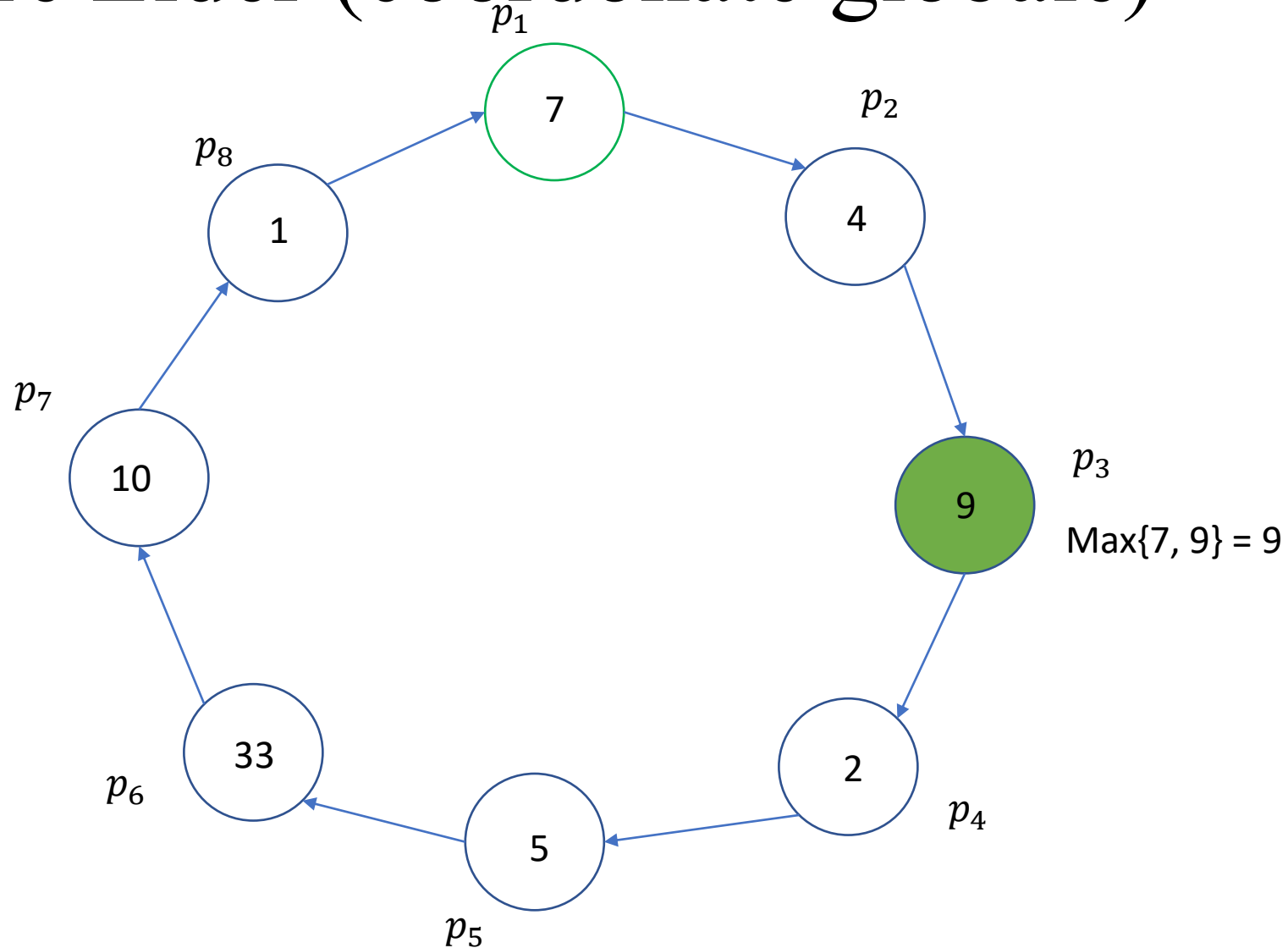
Alegere Lider (coordonate globale)



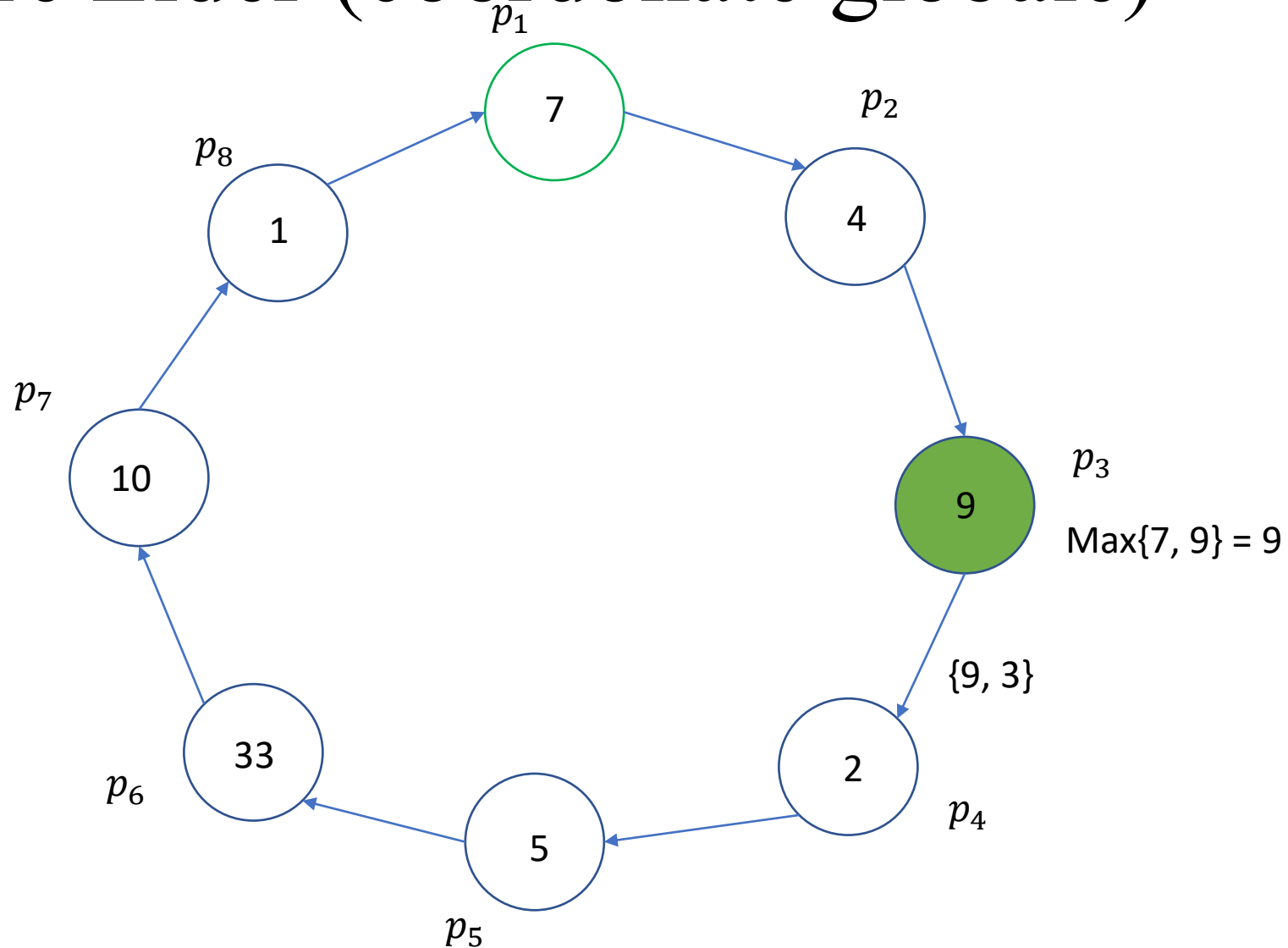
Alegere Lider (coordonate globale)



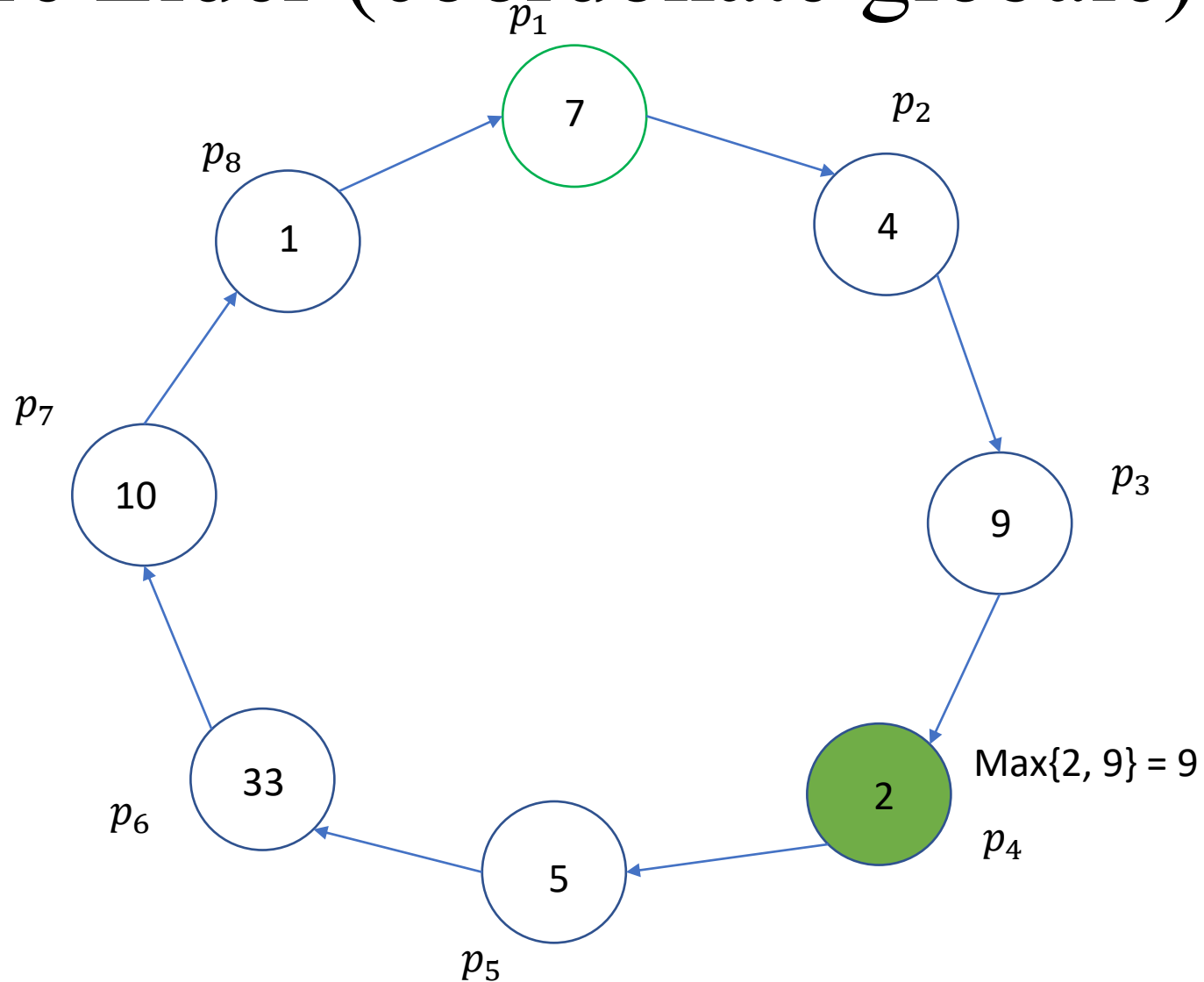
Alegere Lider (coordonate globale)



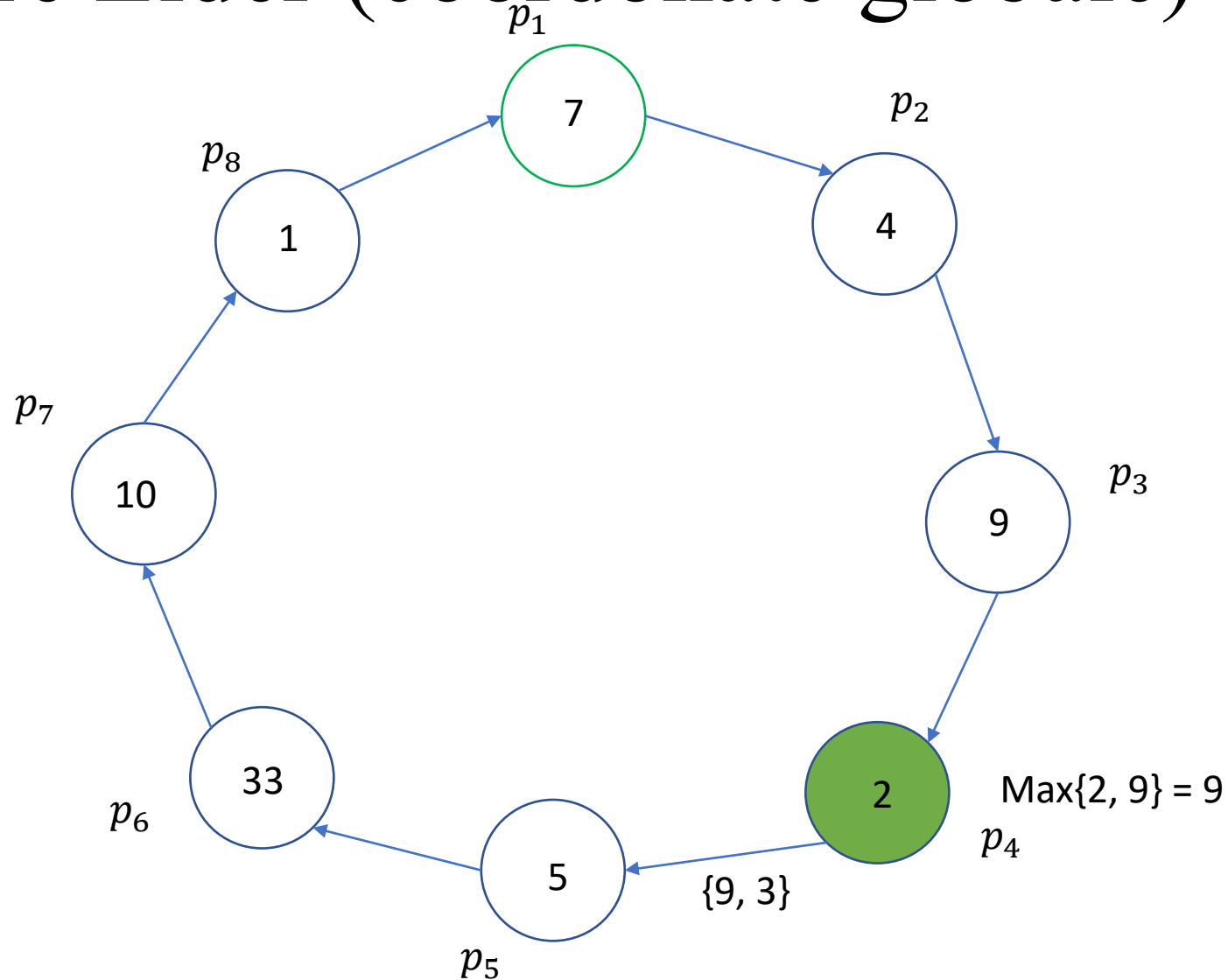
Alegere Lider (coordonate globale)



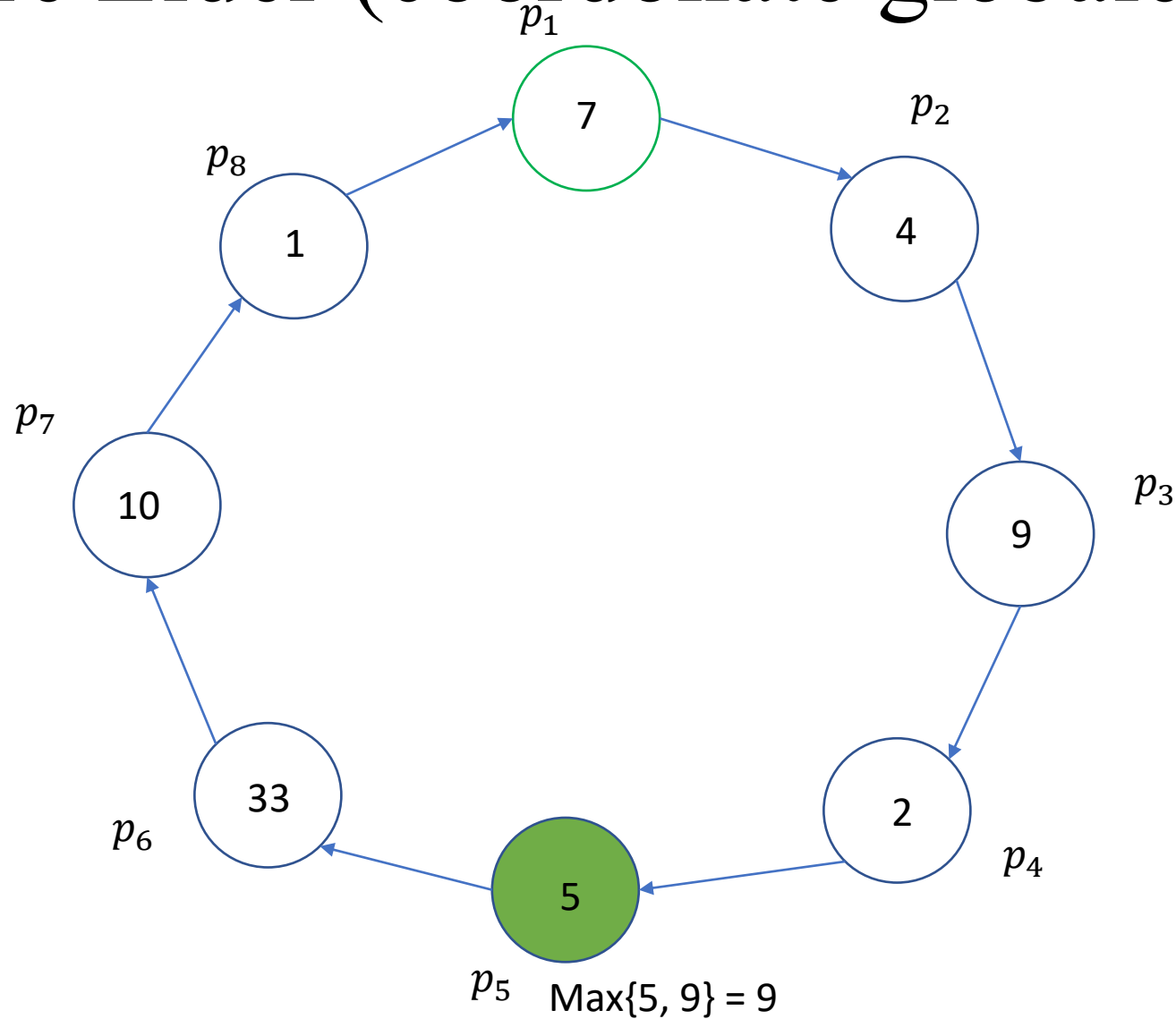
Alegere Lider (coordonate globale)



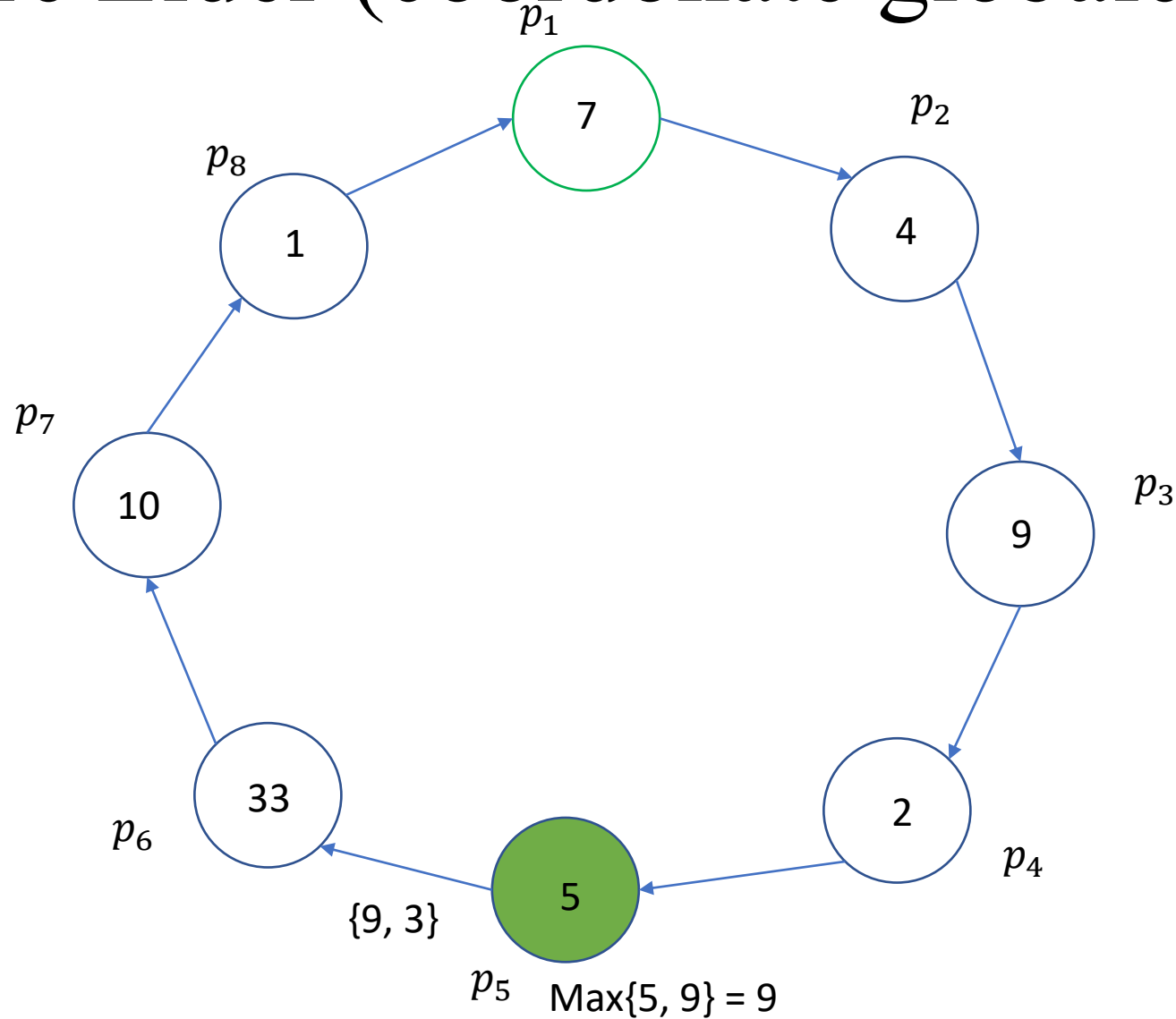
Alegere Lider (coordonate globale)



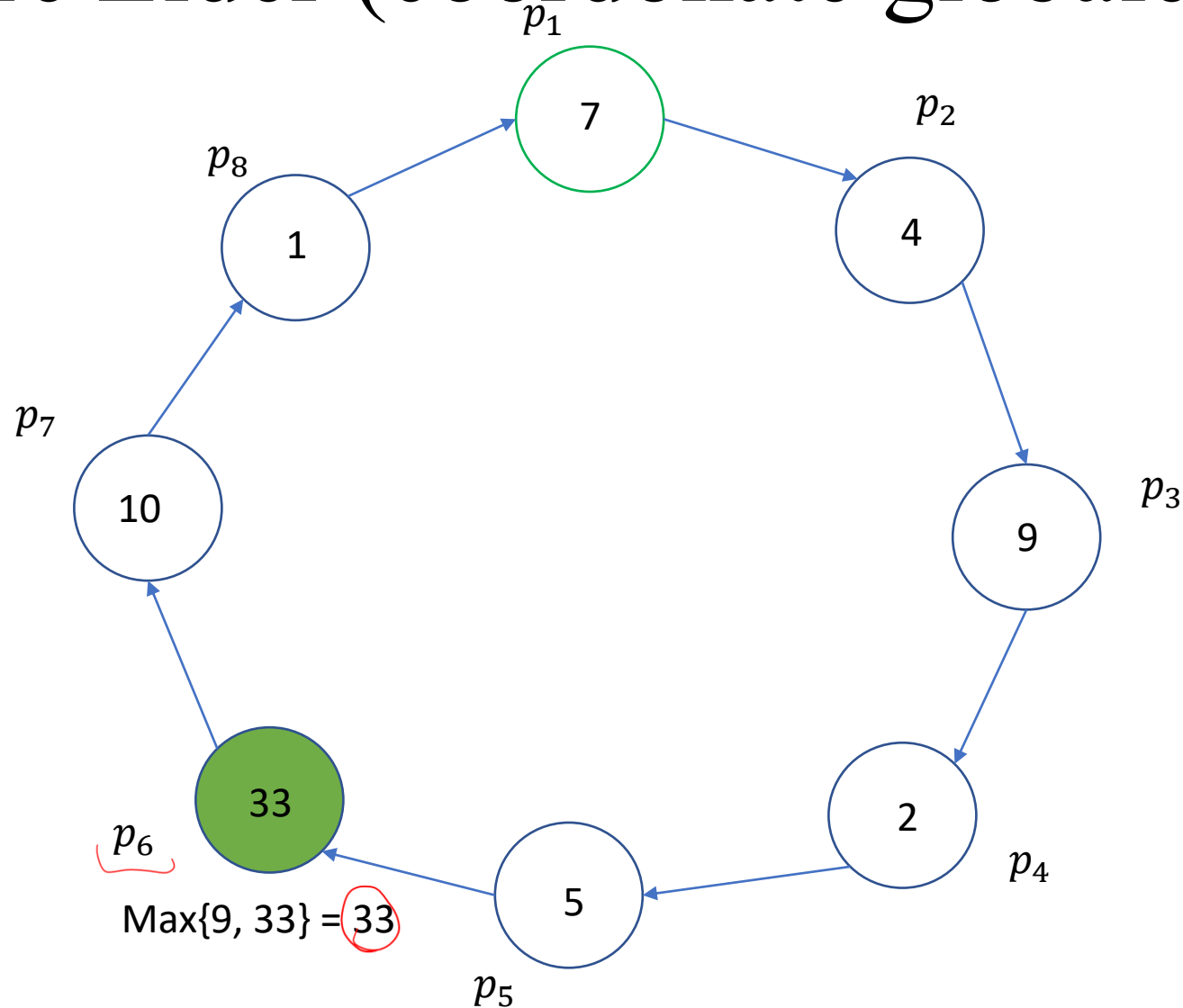
Alegere Lider (coordinate globale)



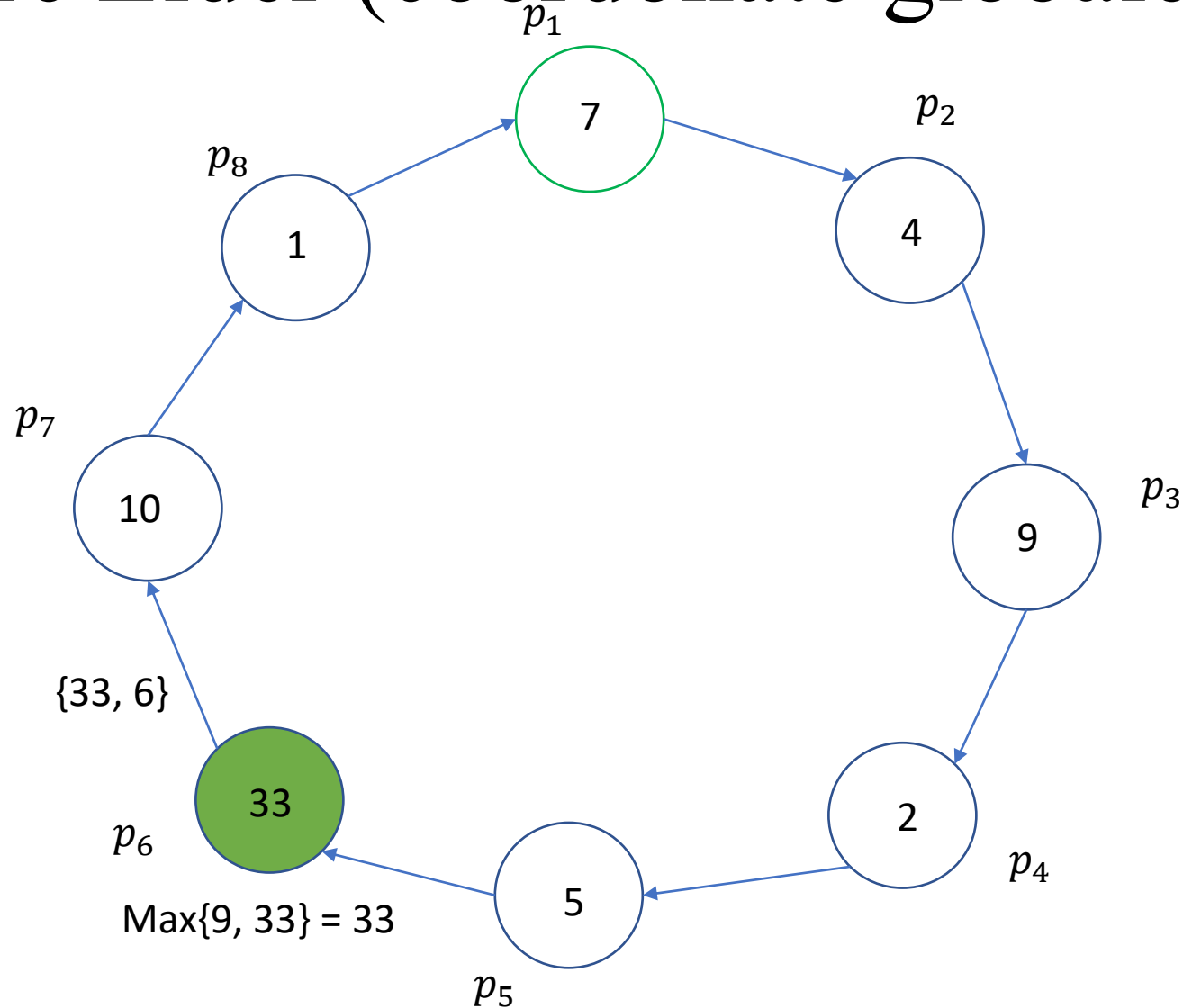
Alegere Lider (coordonate globale)



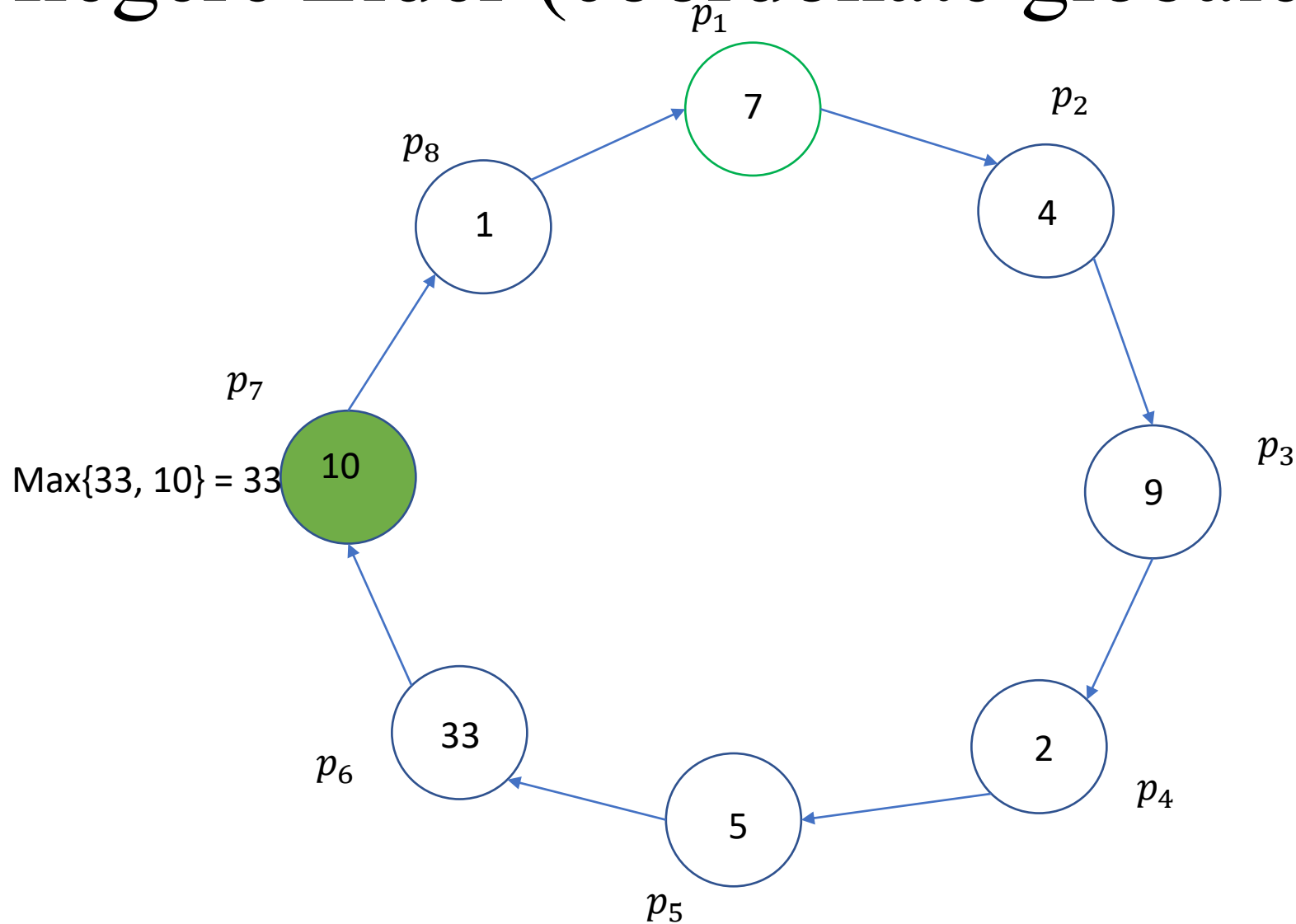
Alegere Lider (coordonate globale)



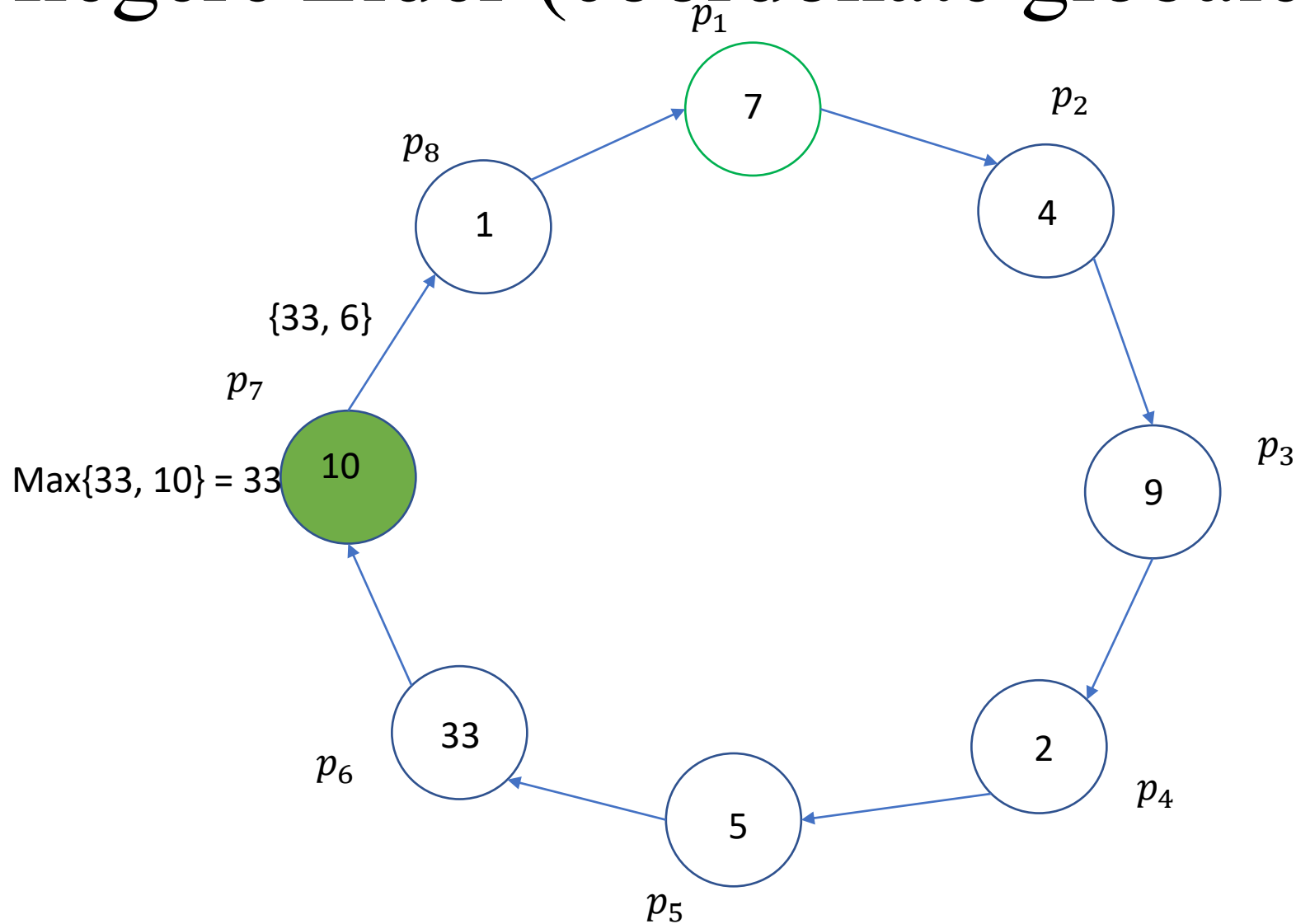
Alegere Lider (coordonate globale)



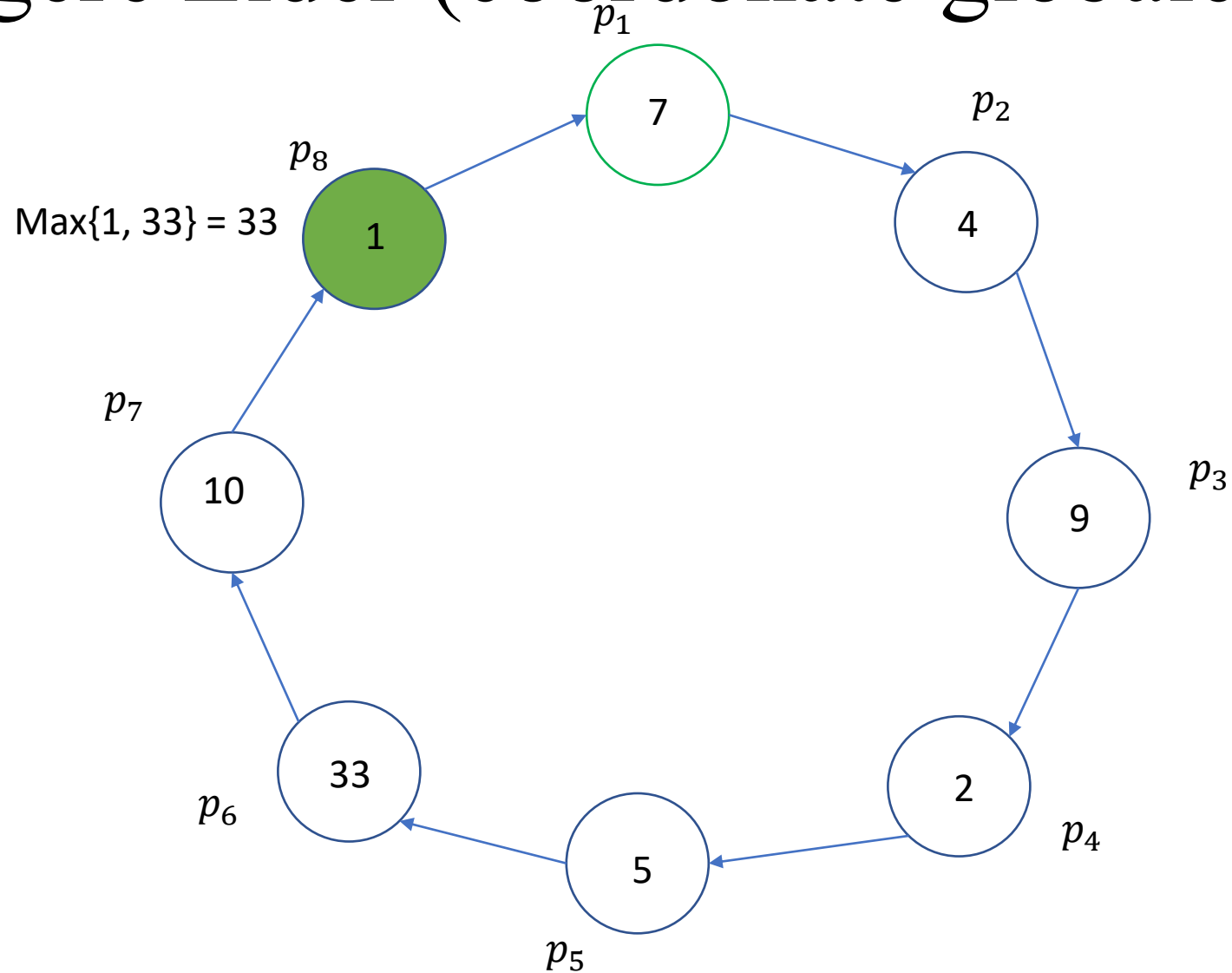
Alegere Lider (coordinate globale)



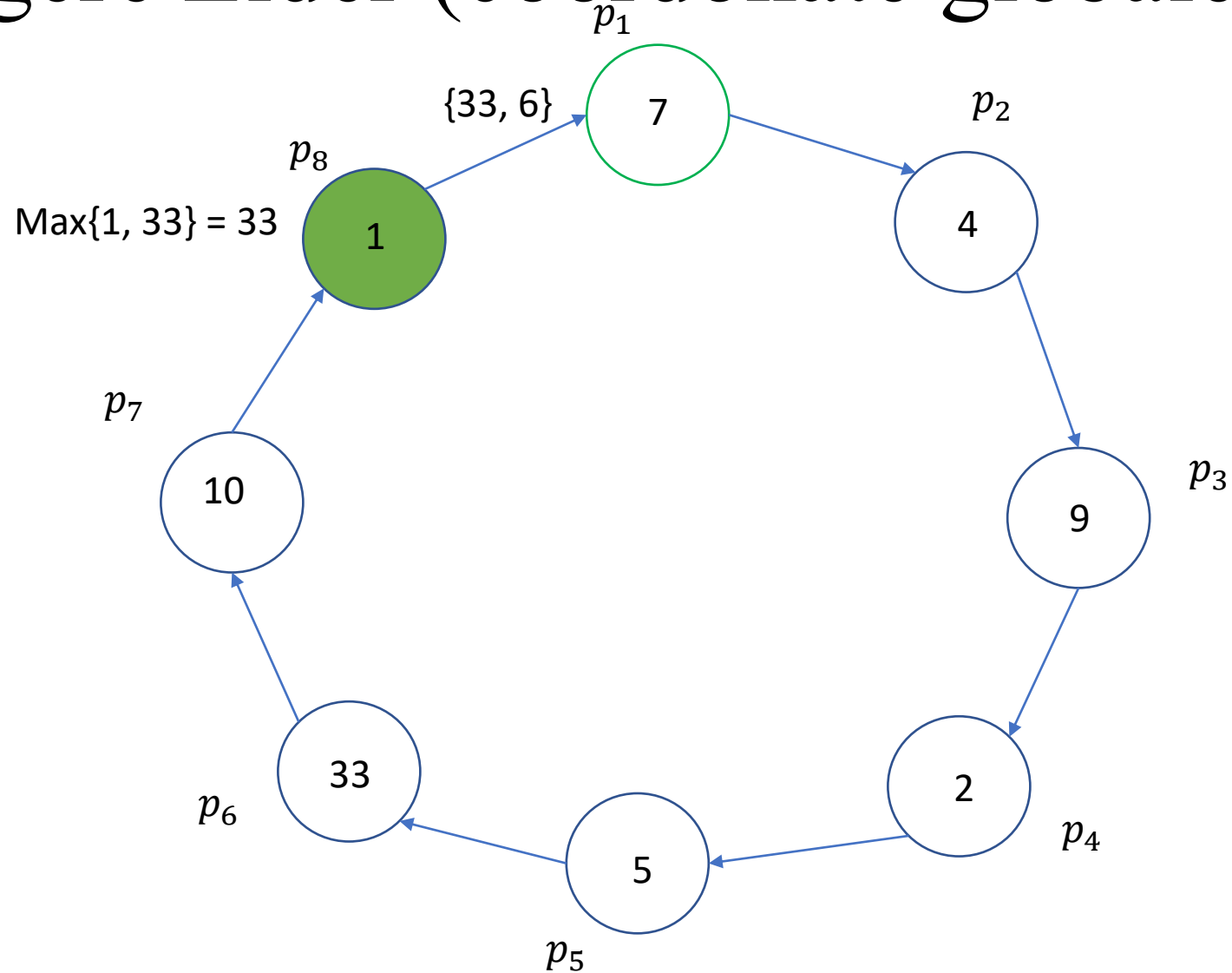
Alegere Lider (coordonate globale)



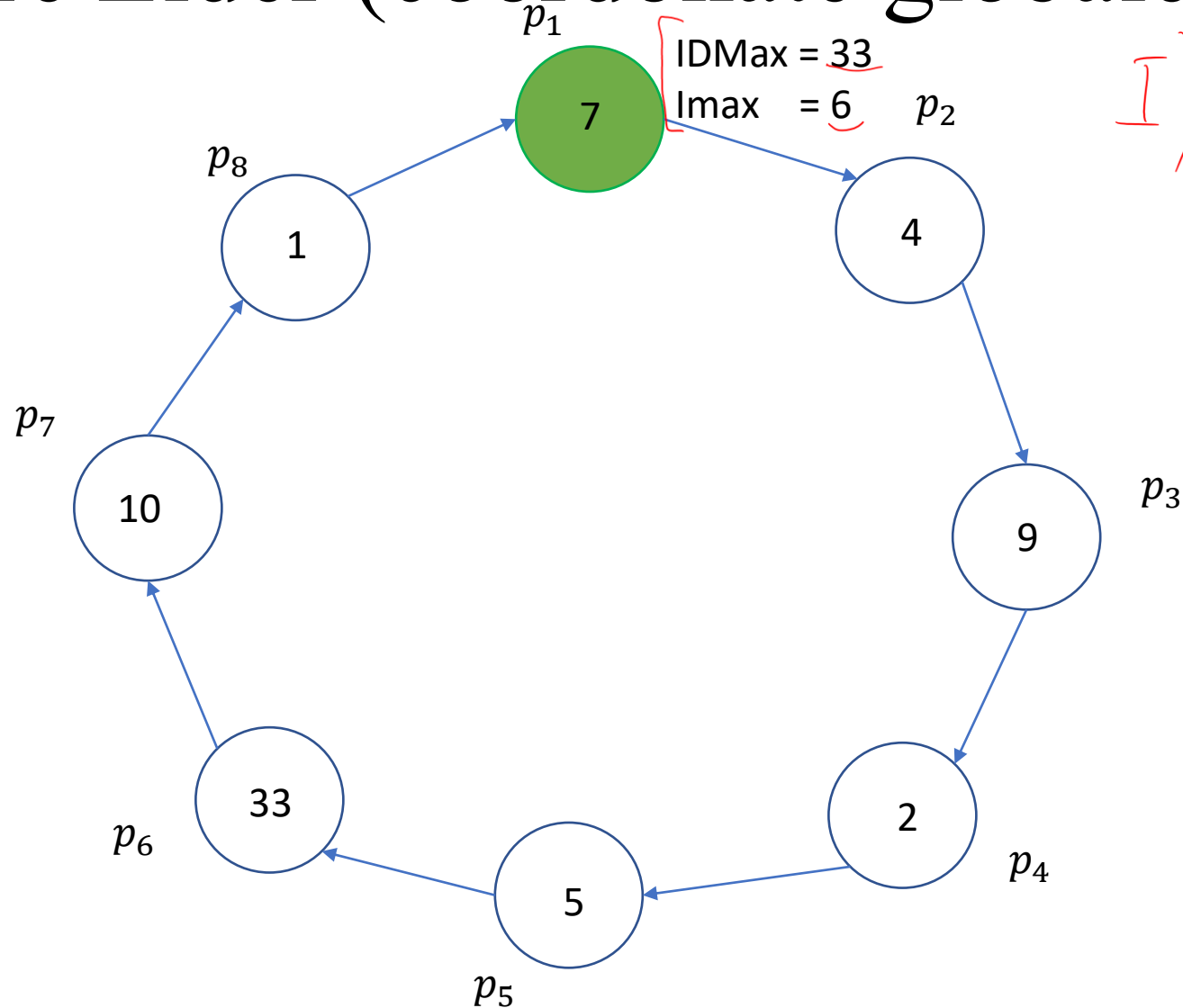
Alegere Lider (coordonate globale)



Alegere Lider (coordonate globale)



Alegere Lider (coordonate globale)



I) Complexitate Timp $O(n)$
Copl. Mesaj $O(n)$

Alegere Lider (coordonate globale)

M_i : - int n (număr noduri)
- int i (index)
- int id (token)
- int id_aux, id_max (memorează id maxim)

II) $Time: O(n)$, $Mesaj: O(n)$
Total: $Time: O(n)$, $Mesaj: O(n)$

% Faza I: Max ID

...

% Faza II: Difuzare Max ID (Broadcast)

Funcție transformare nod i $f_i()$:

1. **If** ($i == 1$):

1. send({idmax, imax}, 2);

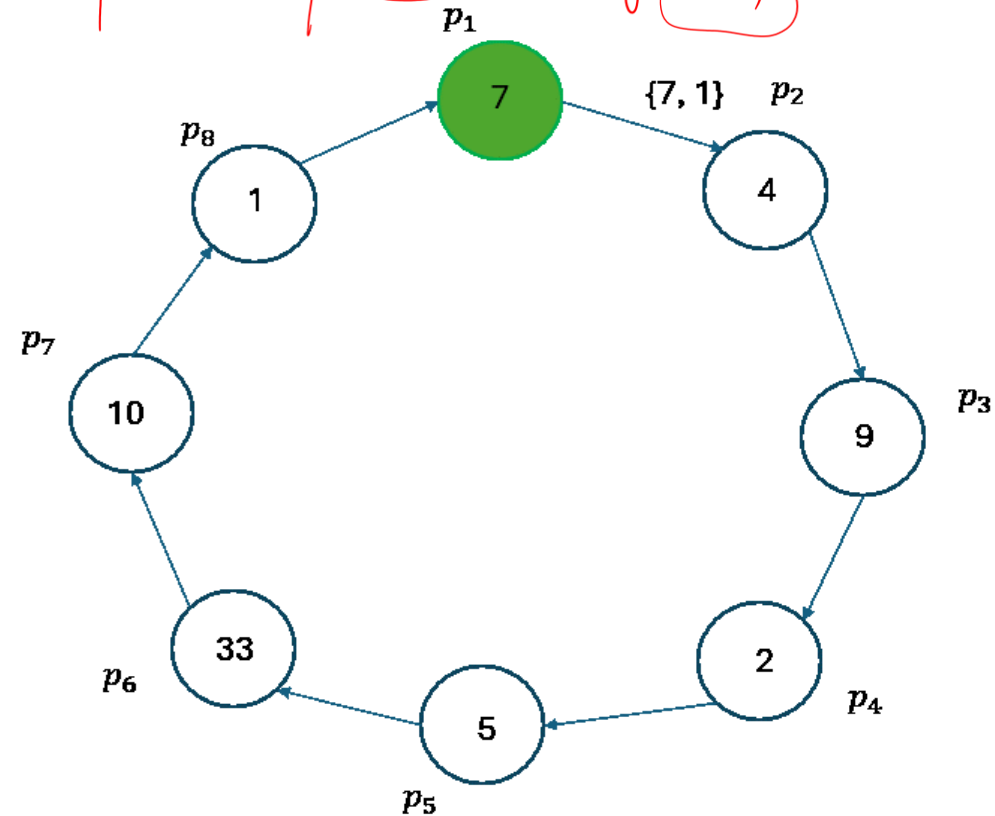
Else If ($i == n-1$):

1. recv({idmax, imax}, $i - 1 \bmod n$);

Else

1. recv({idmax, imax}, $i - 1 \bmod n$);

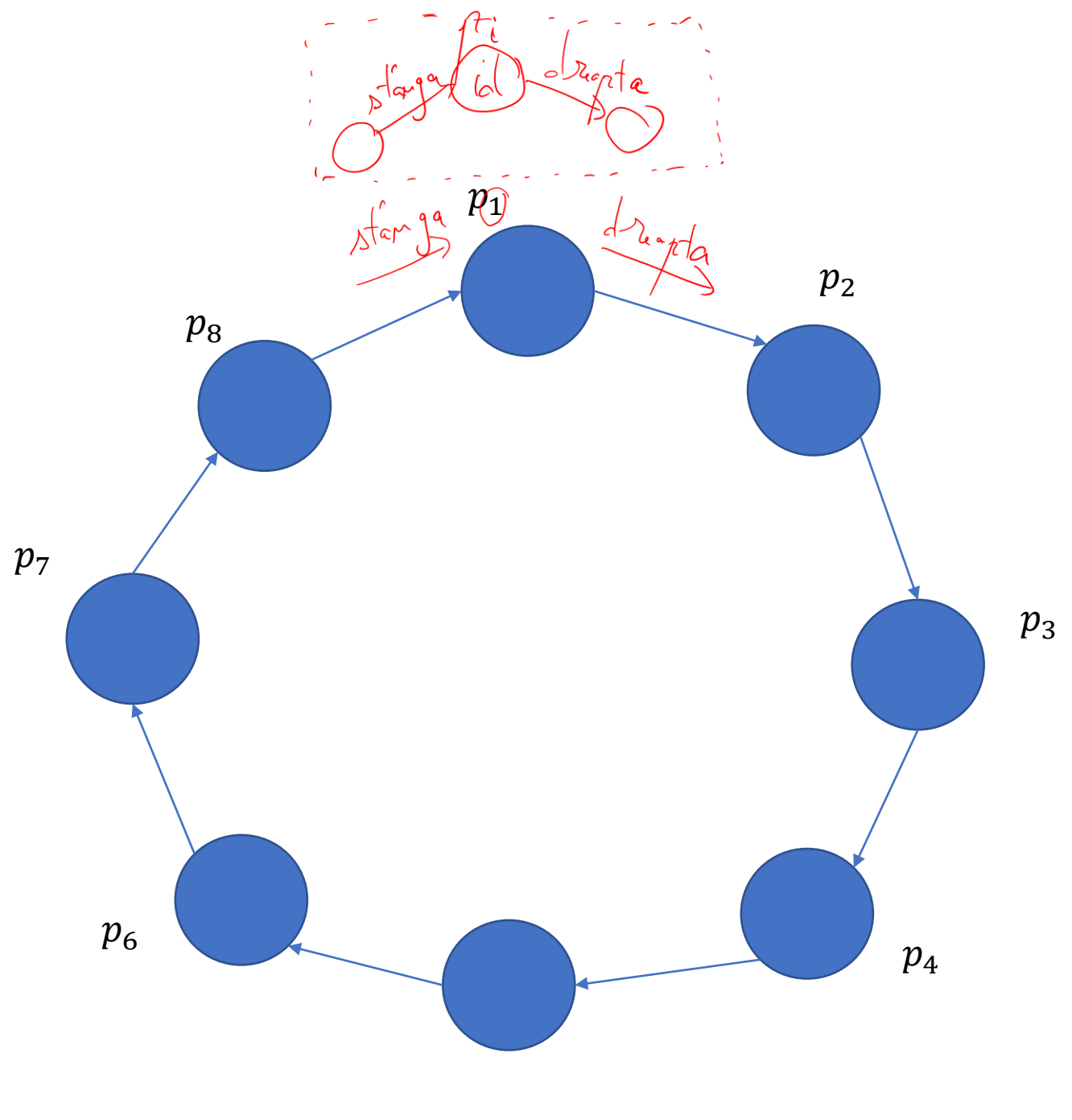
2. send({idmax, imax}, $i + 1 \bmod n$);



Alegere Lider (AL)

Ipoteze: P_i cunoaște

1. Topologie inel unidirecțional
 2. ~~Poziția proprie relativă în topologie~~
(index)
 3. ~~Numărul de noduri n~~
- Ipoteze mai realiste (rețele peer-to-peer)
 - Topologia și numărul de noduri sunt statice
 - Funcția AlegereLiderInel() implică indexul local și numărul de noduri!



Algoritmul Flooding (LCR)

Algoritm **Flooding**():

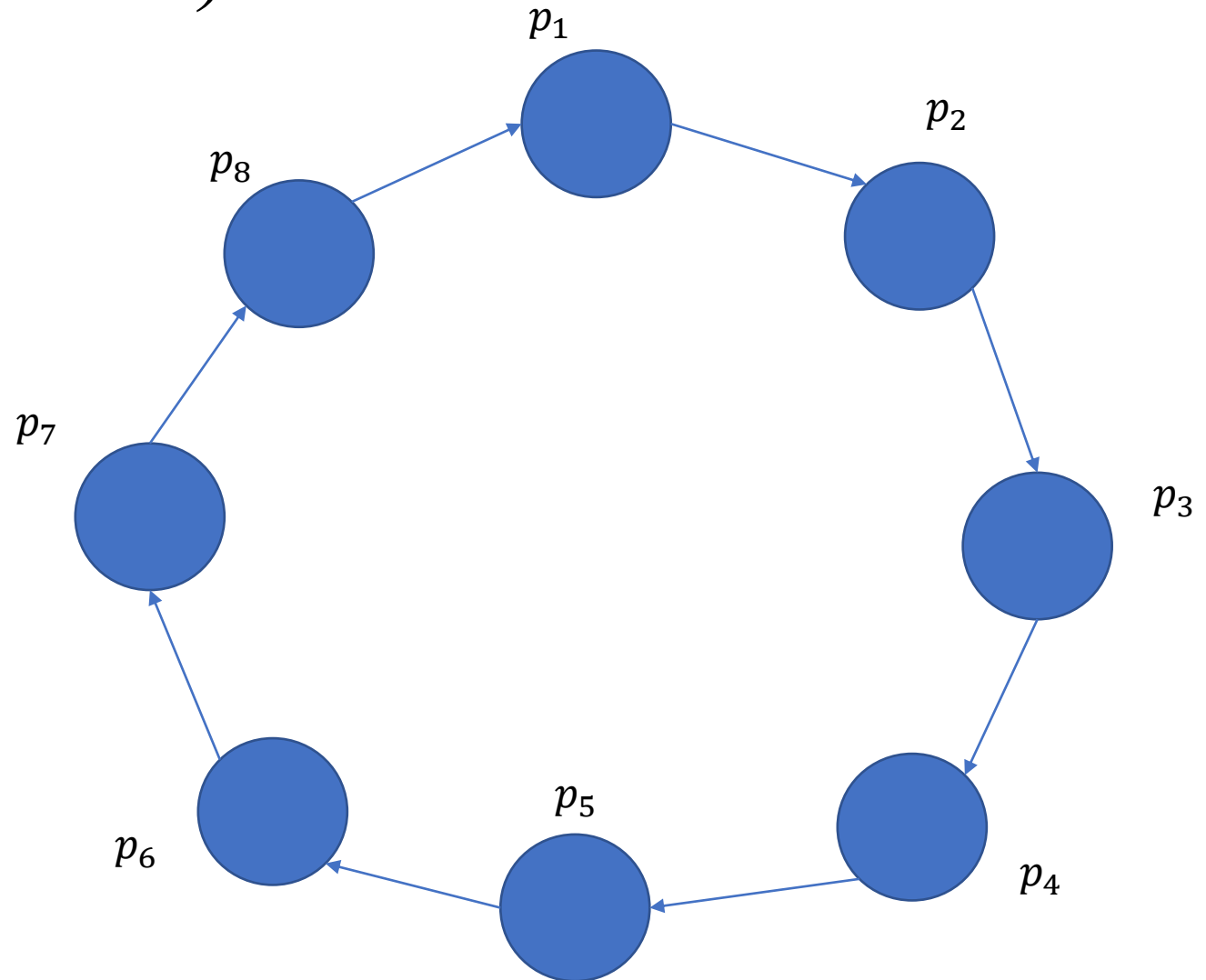
M_i : - int id (token id local)

- int send_id, recv_id (var auxiliare), inițial id
- status $\in \{\text{lider, non-lider}\}$, inițial non-lider

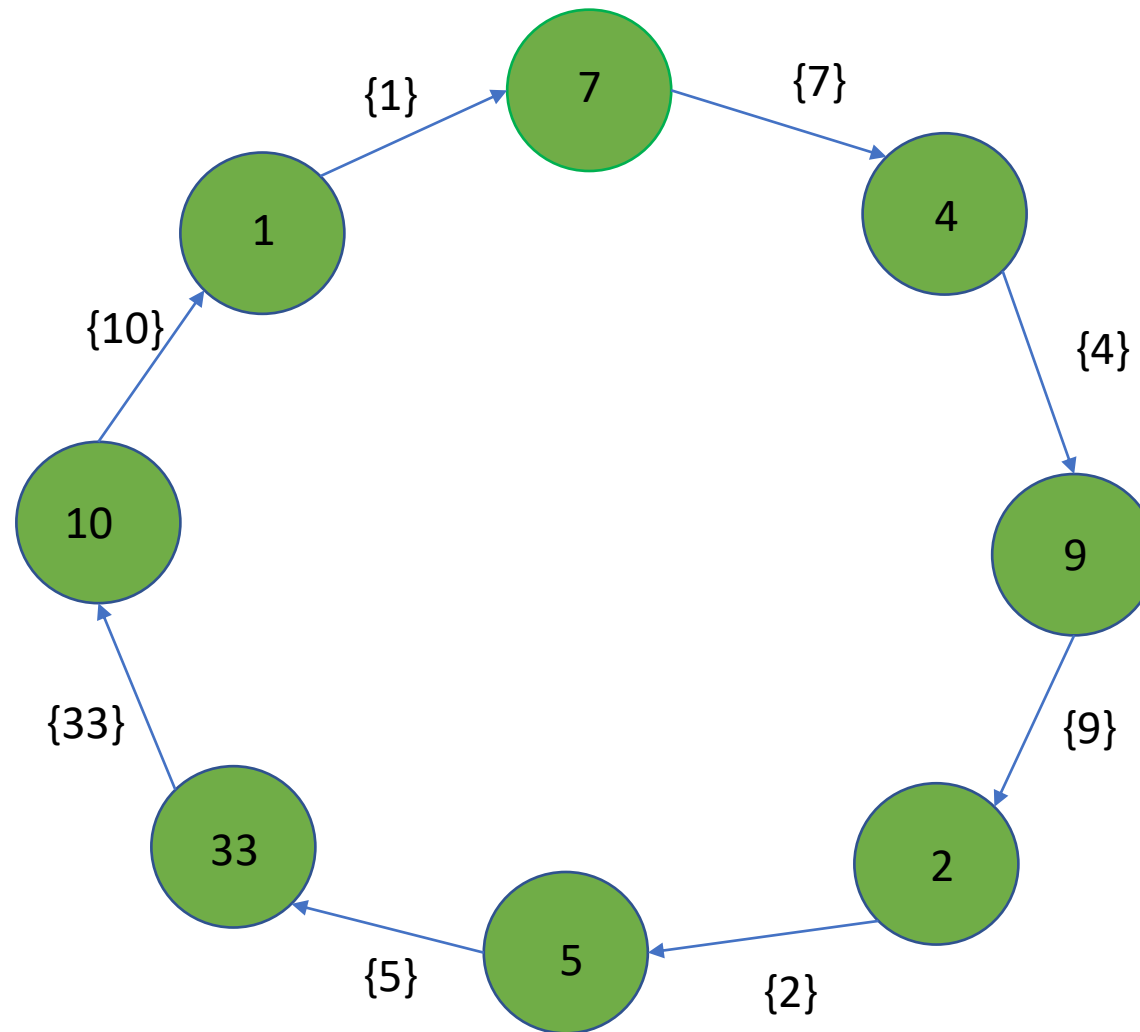
Funcție transformare nod i ():

1. send(send_id, dreapta);
2. recv(recv_id, stânga);
3. **If** (recv_id > id):
 1. send_id := recv_id;
4. **ElseIf** (recv_id == id): status = leader;
5. **ElseIf** (recv_id < id): send_id := NULL;

1 mesaj

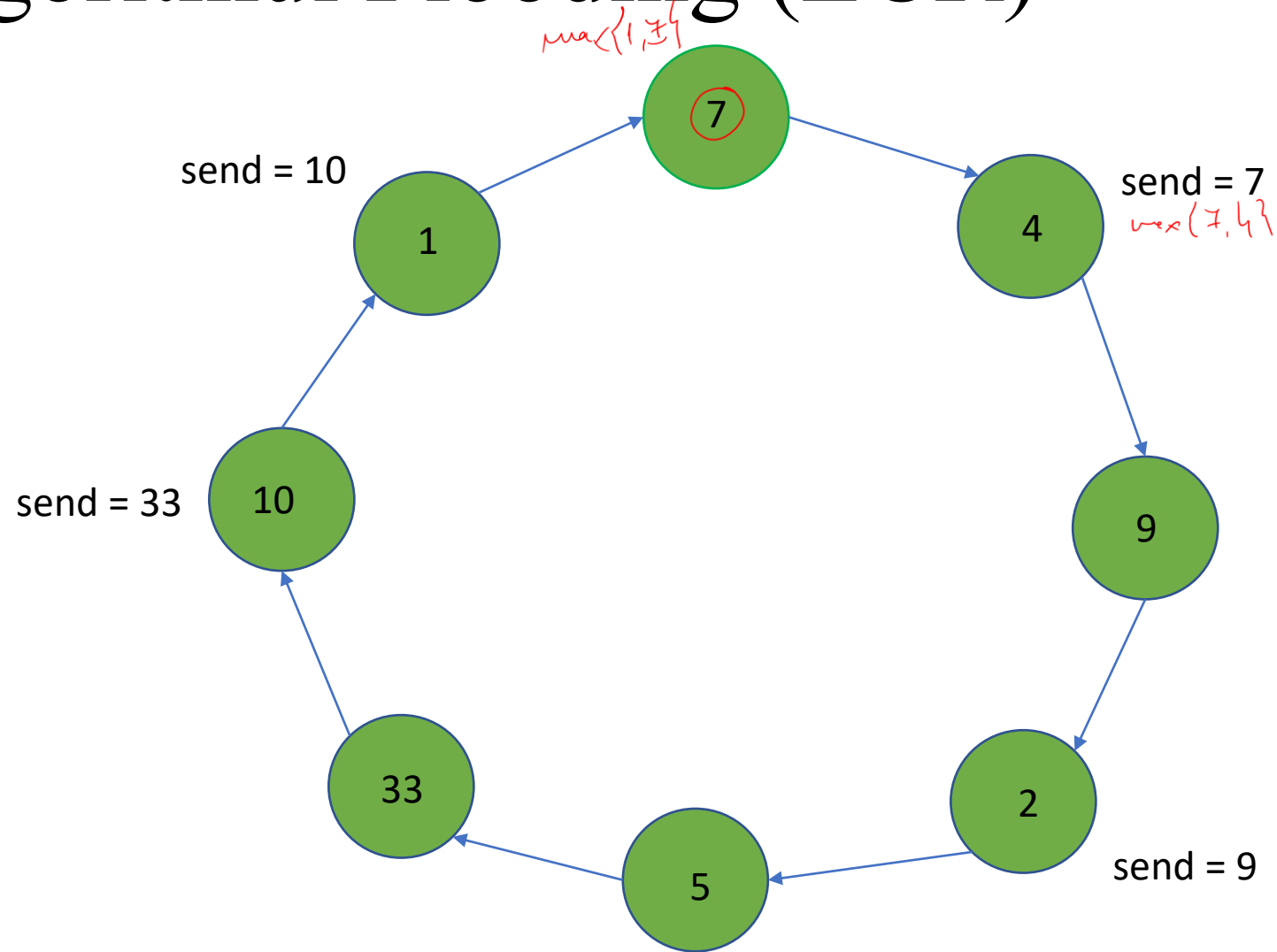


Algoritmul Flooding (LCR)



Iterația 1 $x(1)$

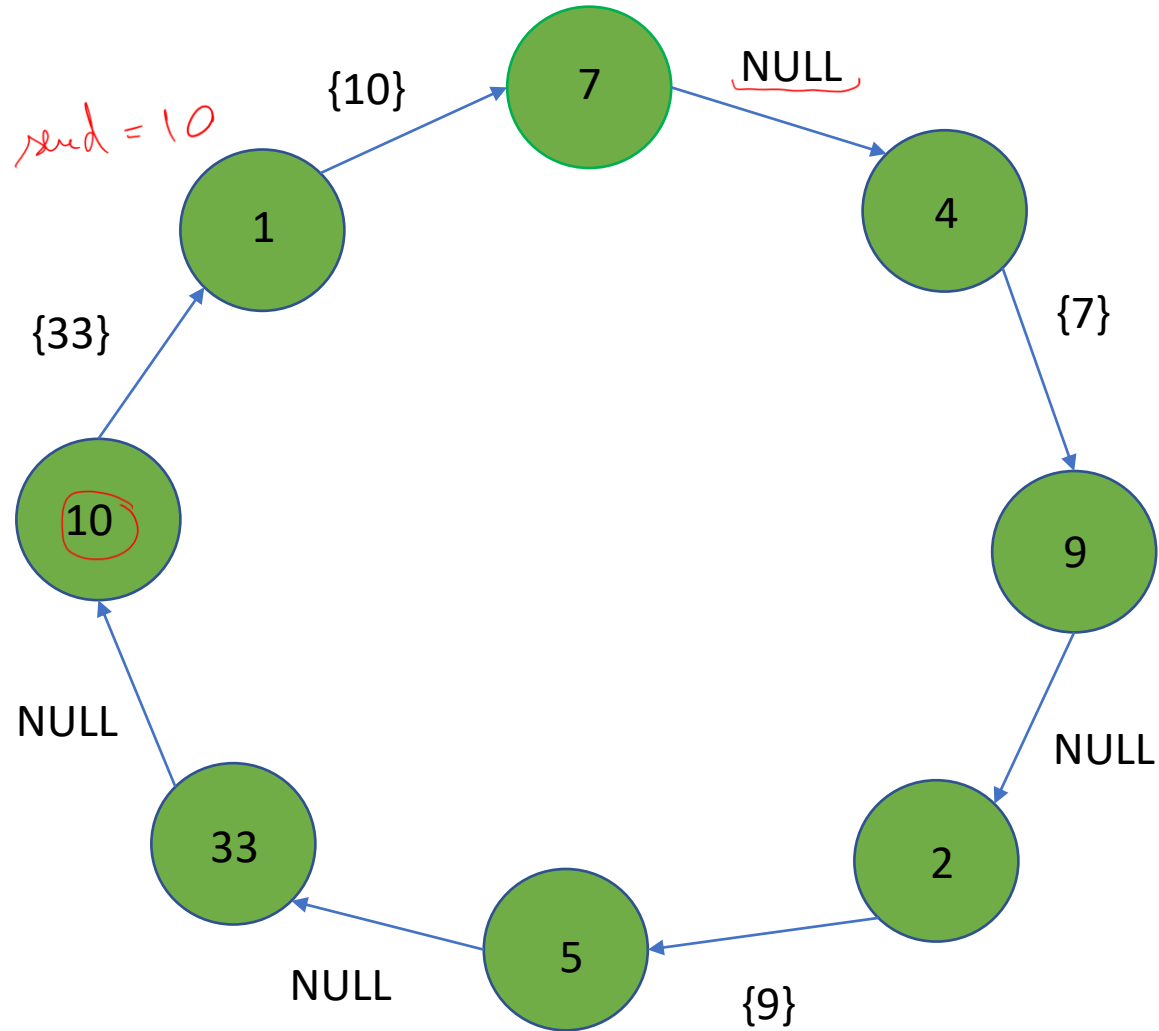
Algoritmul Flooding (LCR)



Iterația 1 $x(1)$

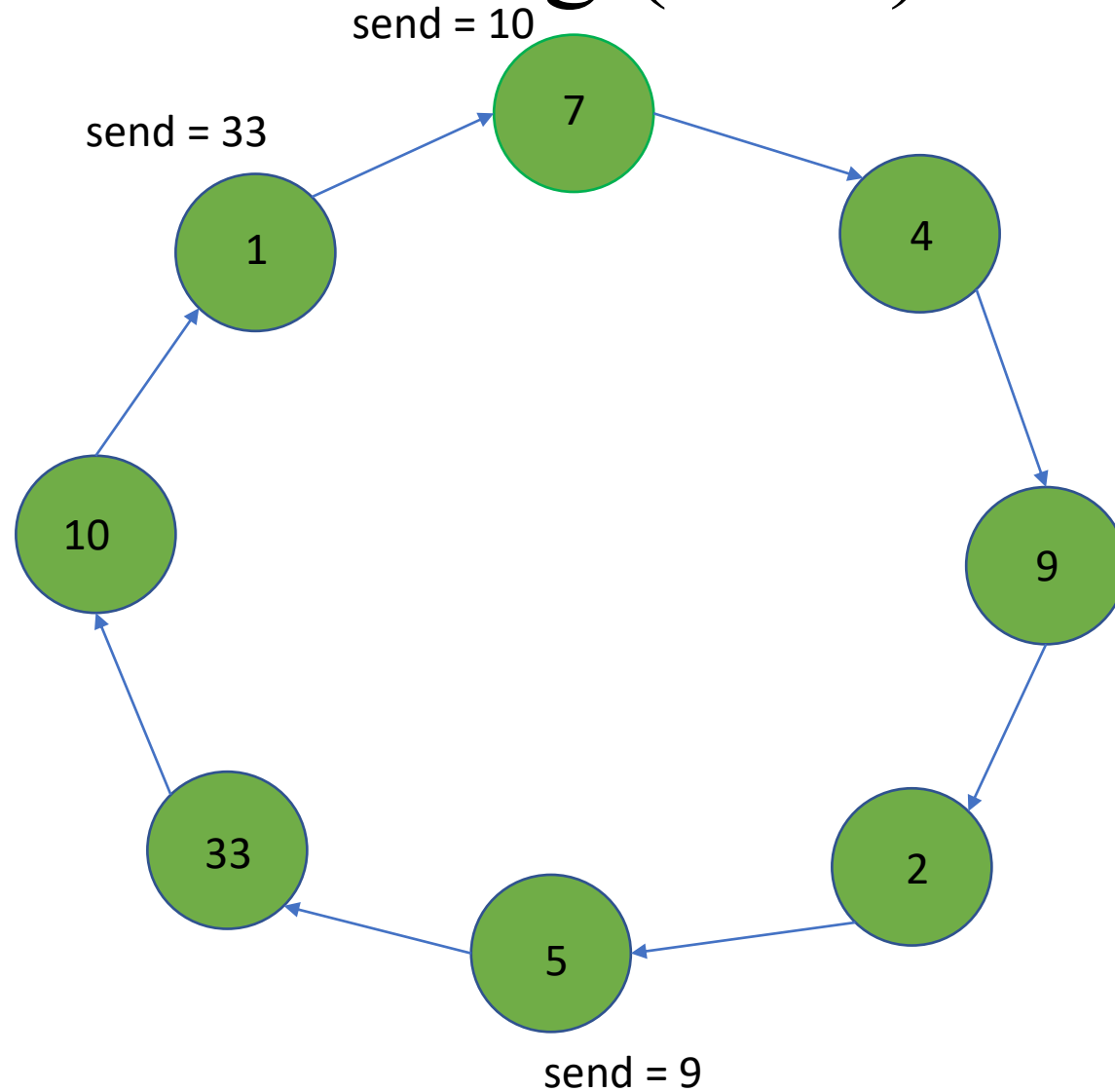
$$\max\{\underline{id}, \underline{recv_id}\}$$
$$\boxed{\text{send_id}} = \text{recv_id}$$

Algoritmul Flooding (LCR)



Iterația 2 $x(2)$

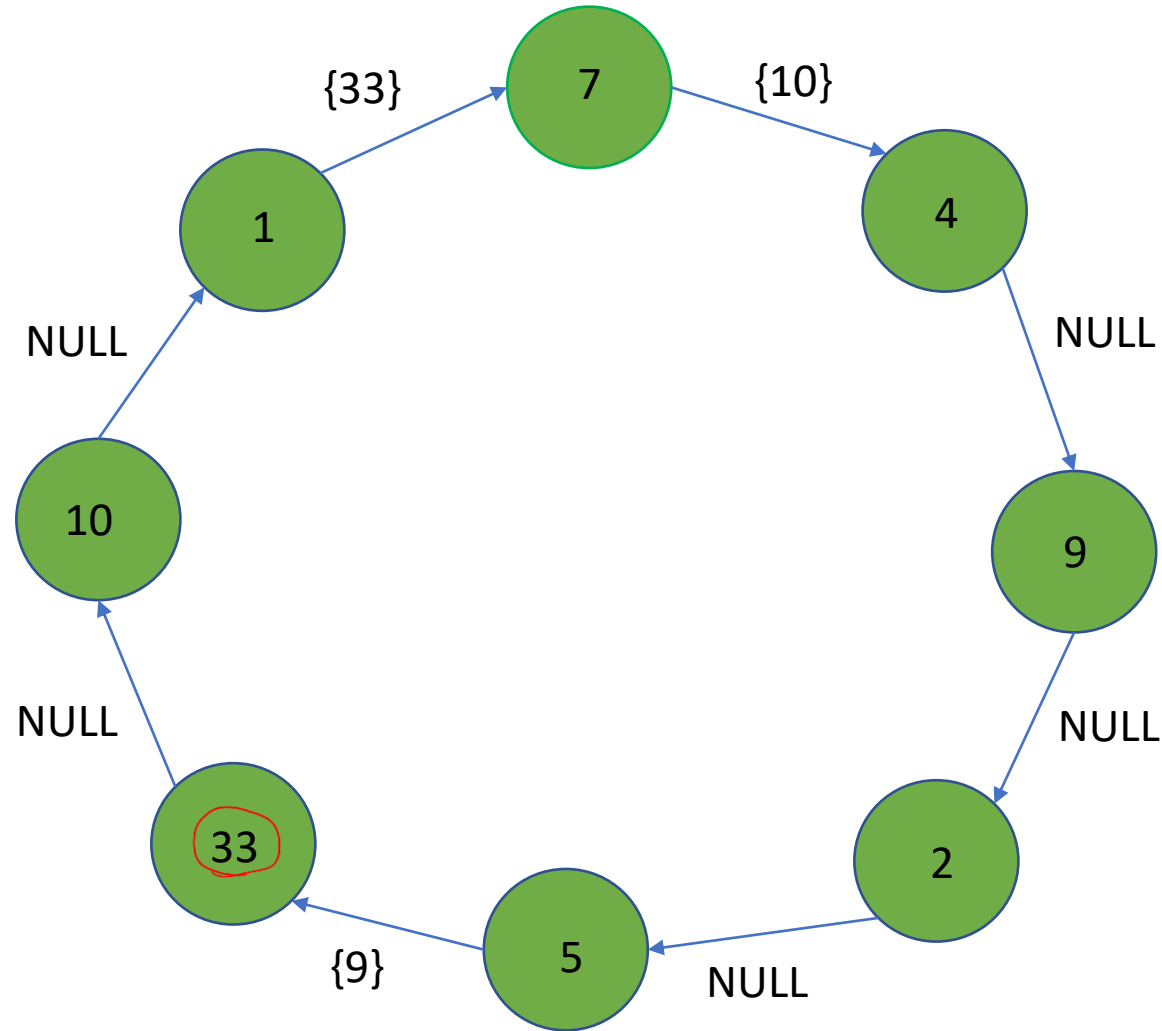
Algoritmul Flooding (LCR)



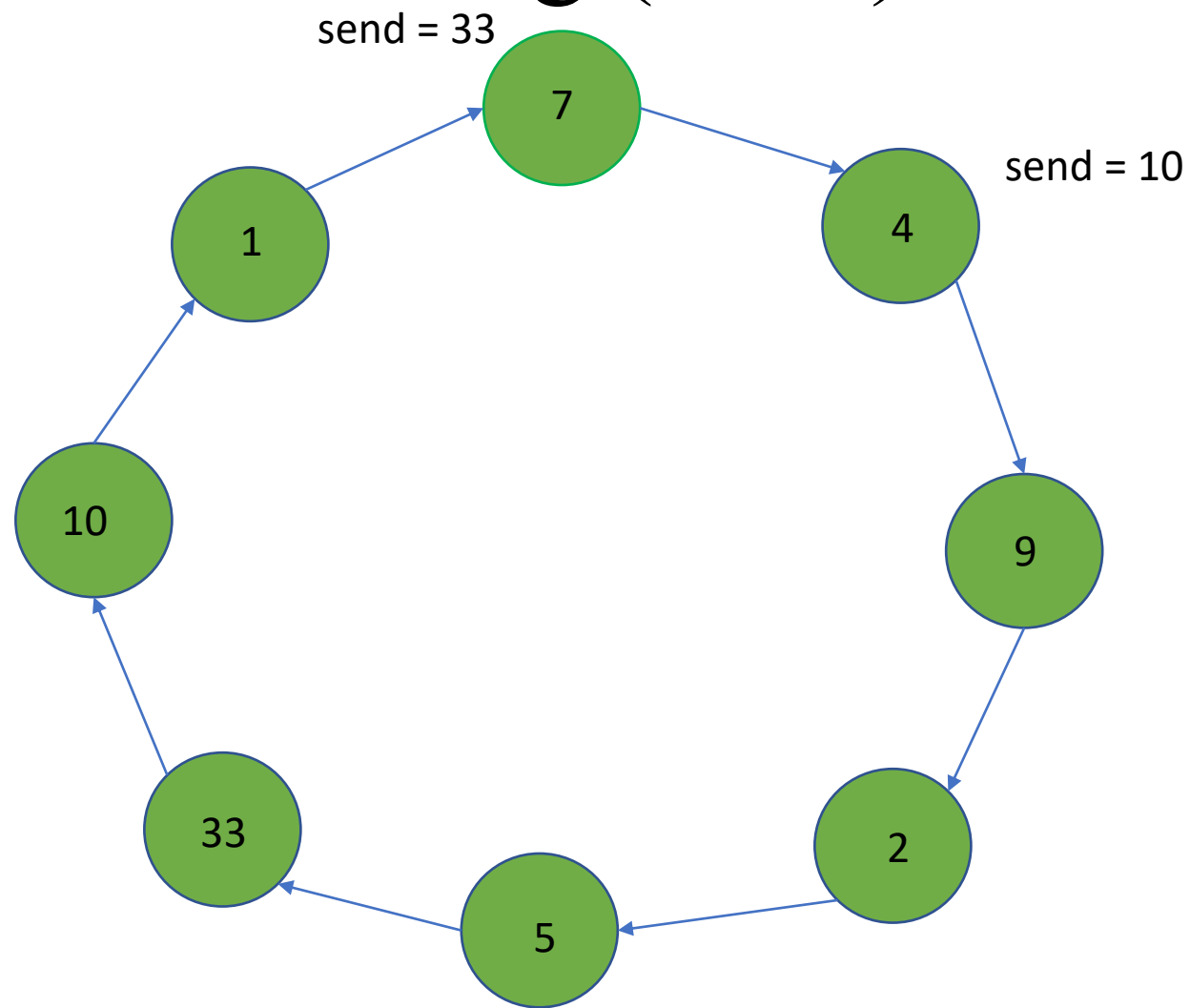
Iterația 2 $x(2)$

Algoritmul Flooding (LCR)

Iterația 3 $x(3)$



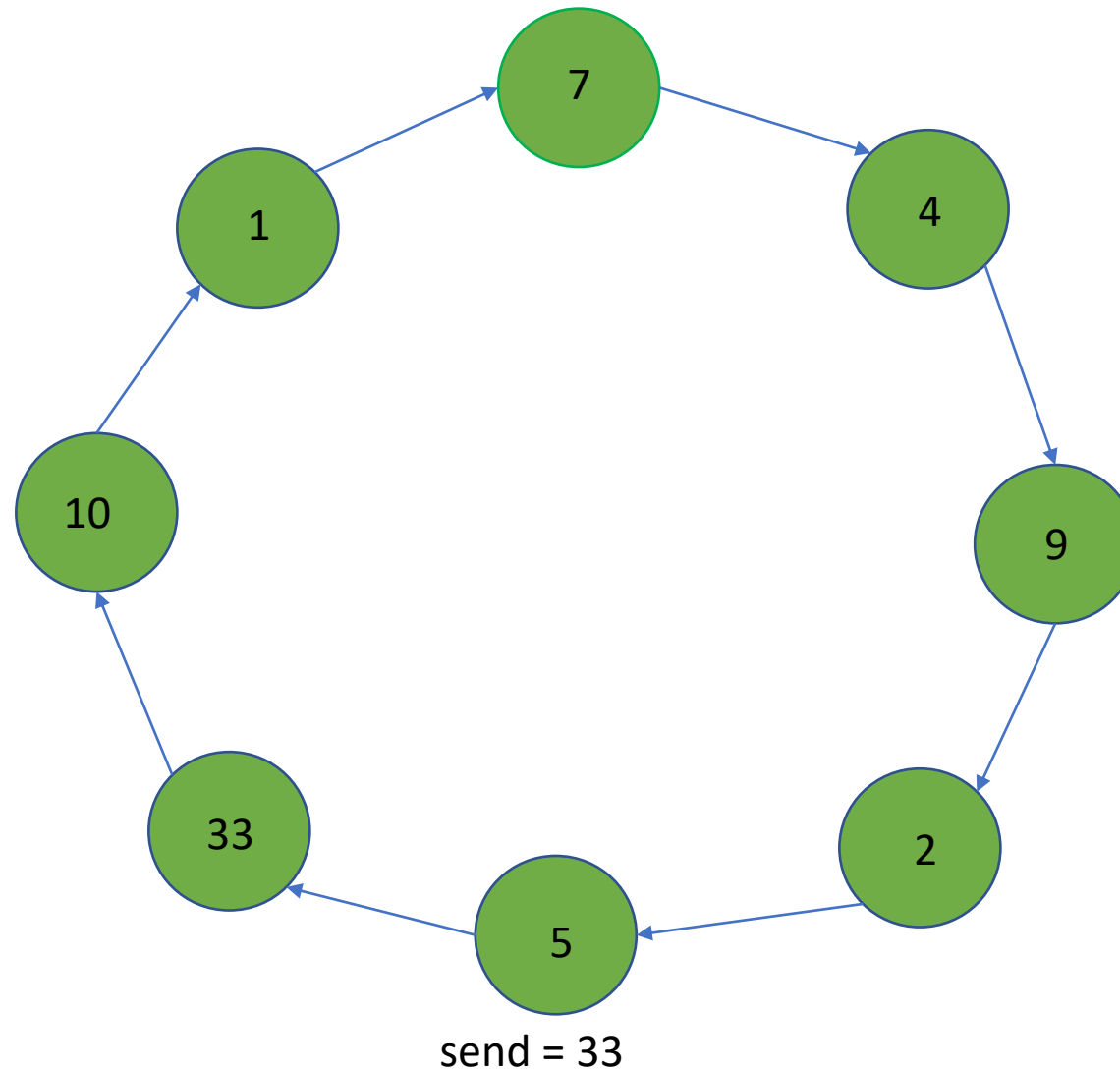
Algoritmul Flooding (LCR)



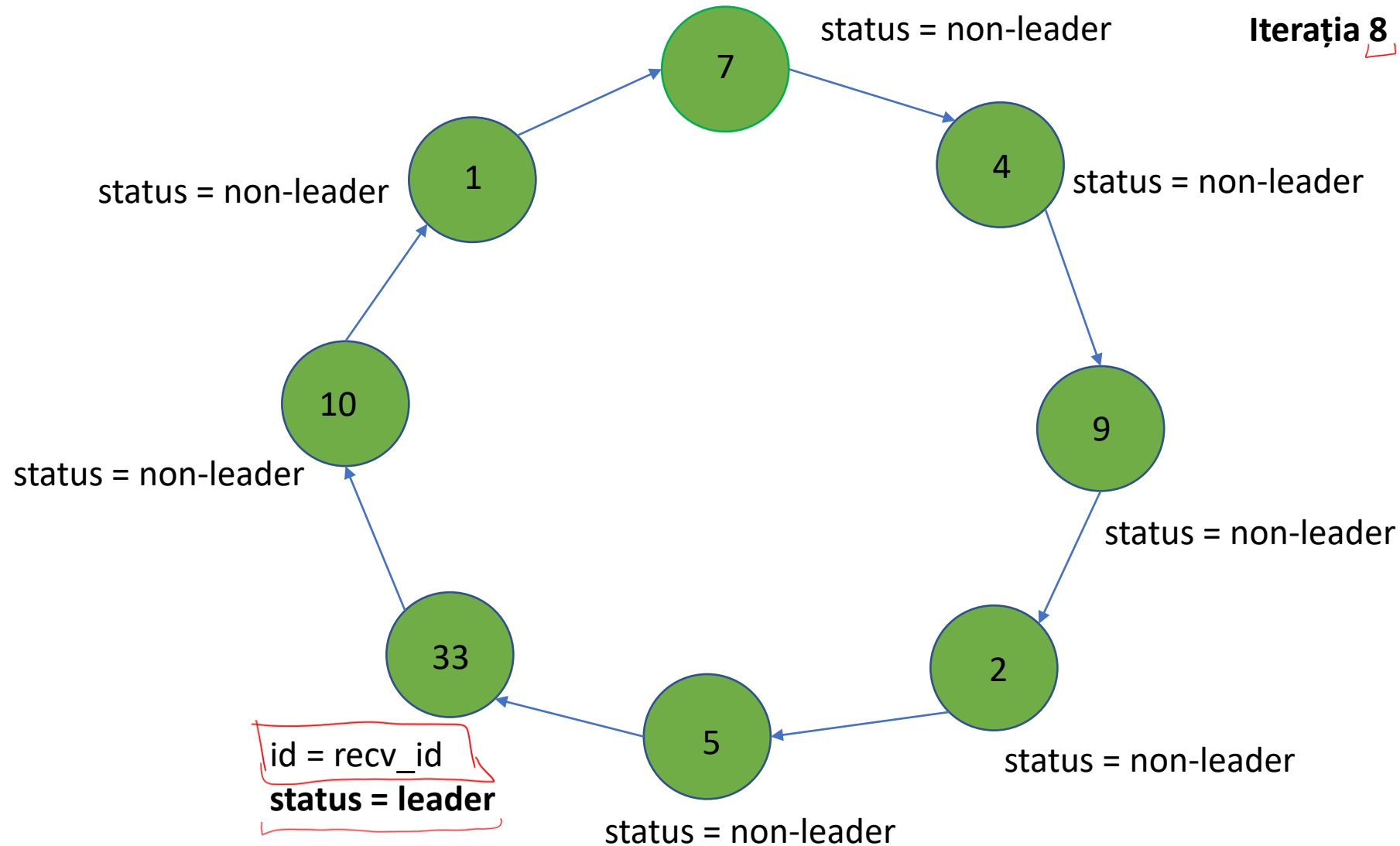
Iterația 3 $x(3)$

Algoritmul Flooding (LCR)

Iterația 7 $x(7)$



Algoritmul Flooding (LCR)



Iterația 8 $x(8)$

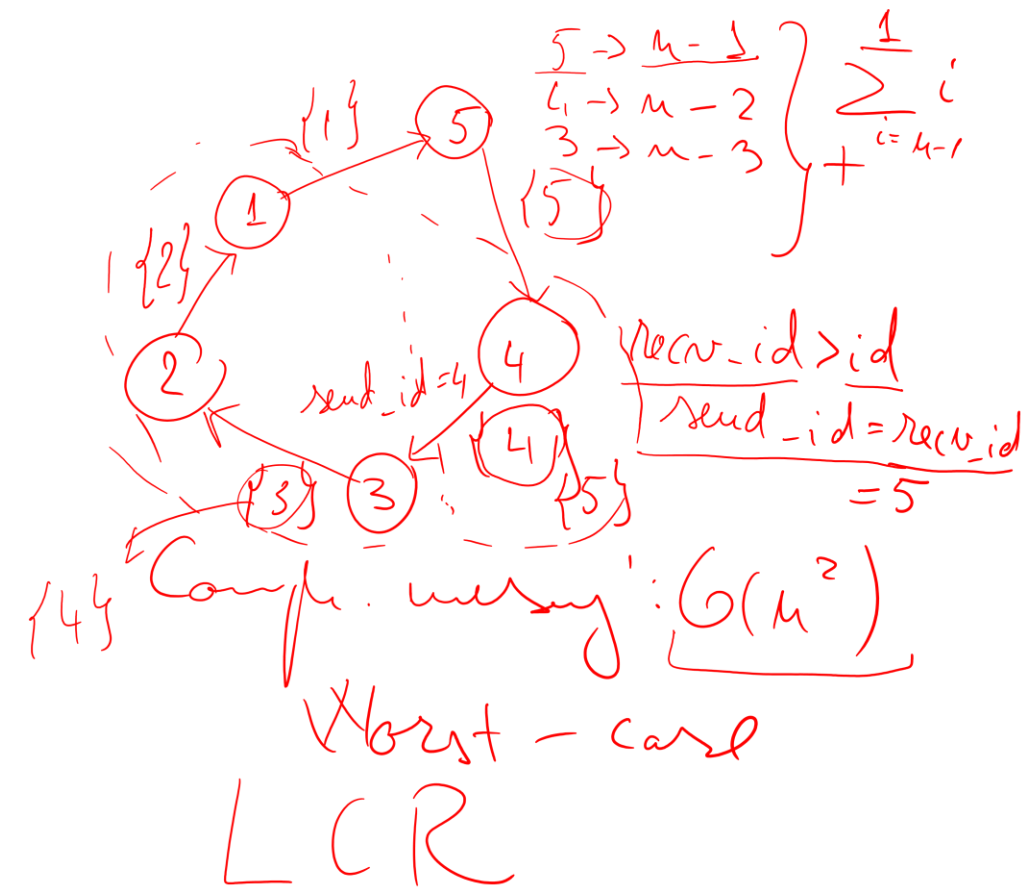
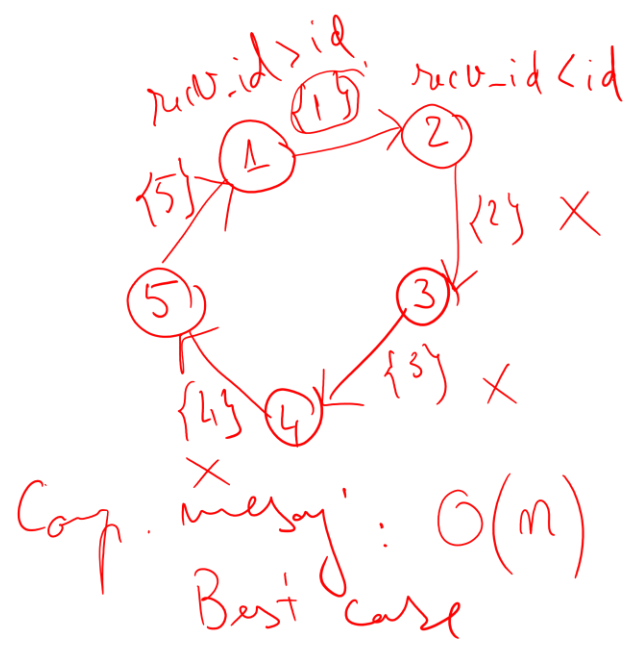
Complexitate timp?

$O(n)$

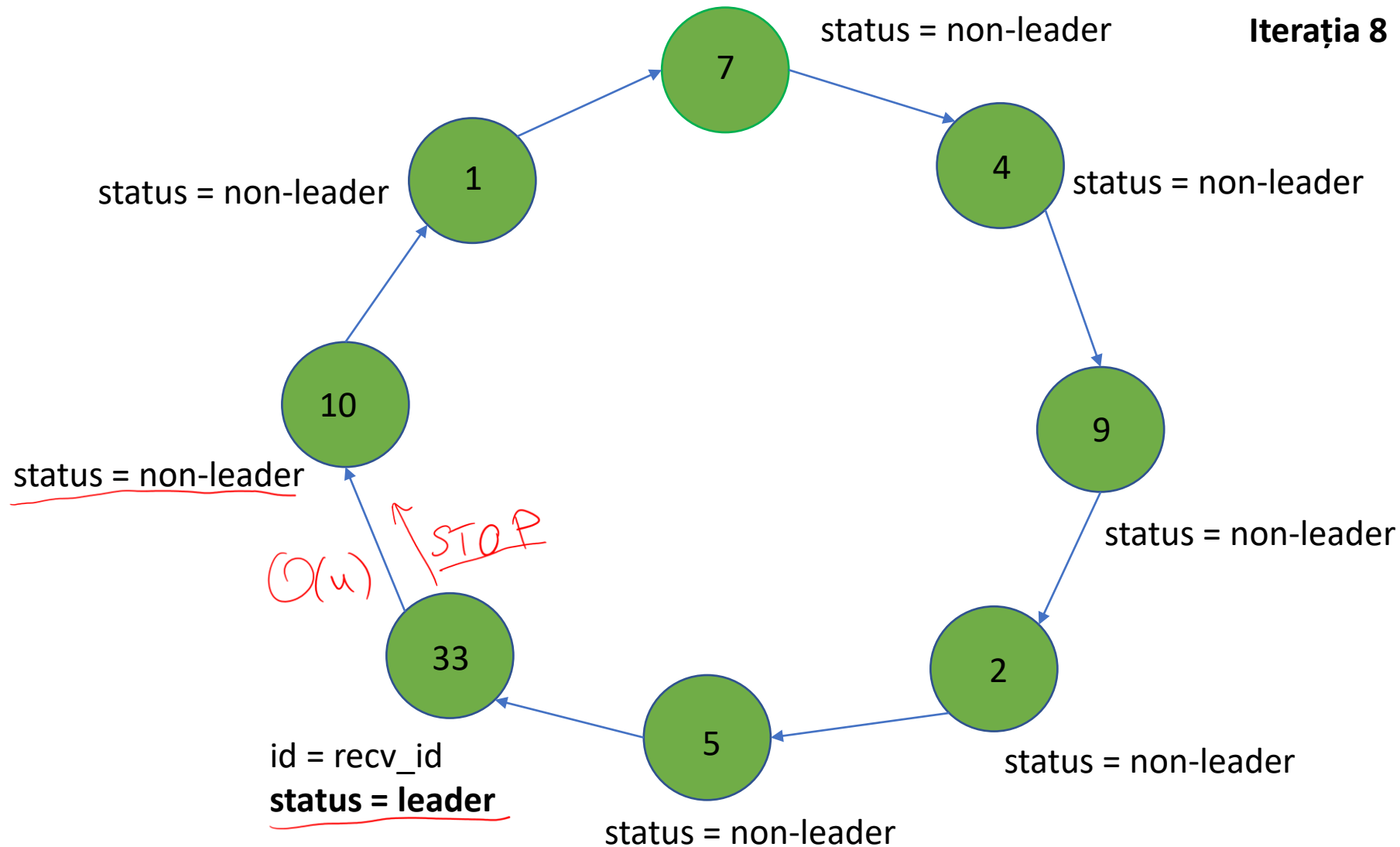
Complexitate mesaj?

$O(1 \cdot n \cdot n)$ worst case
 n^2

Optimal: $O(n \log(n))$
incl



Algoritmul Flooding (LCR)



- In varianta curentă nu avem criteriu de oprire (nodurile execută iterația permanent)

- **Soluție:** Pentru oprire liderul poate difuza un mesaj de raport, prin care semnalează încheierea competiției

Algoritmul Flooding (LCR)

$\left. \begin{array}{l} j \neq i_{\max} \Rightarrow \text{nou lider} \\ i_{\max} \Rightarrow \text{lider} \end{array} \right\} \text{stabile}$

Algoritm **FloodingLCR()**:

M_i : - int id (token id local)

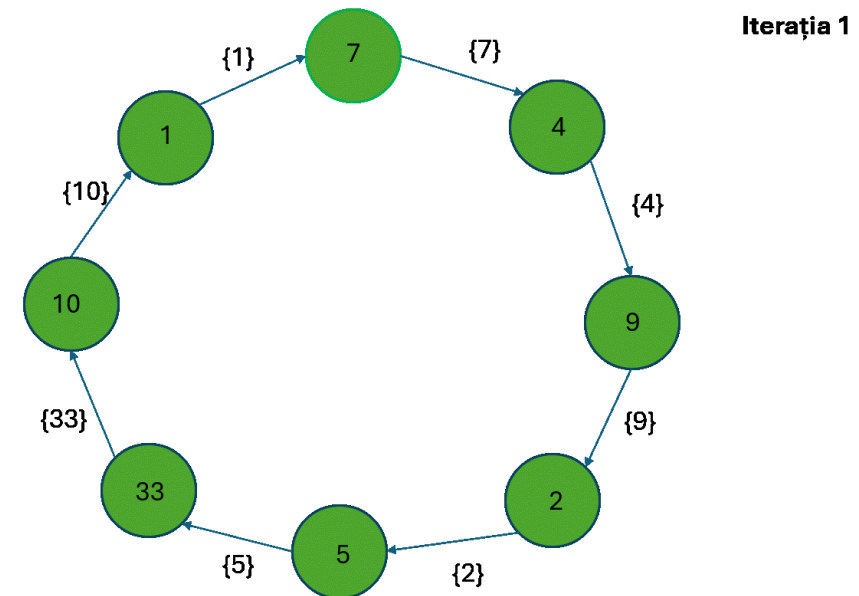
- int send_id, rcv_id (var auxiliare), inițial id
- status $\in \{\text{lider, non-lider}\}$, inițial non-lider

Funcție transformare nod i ():

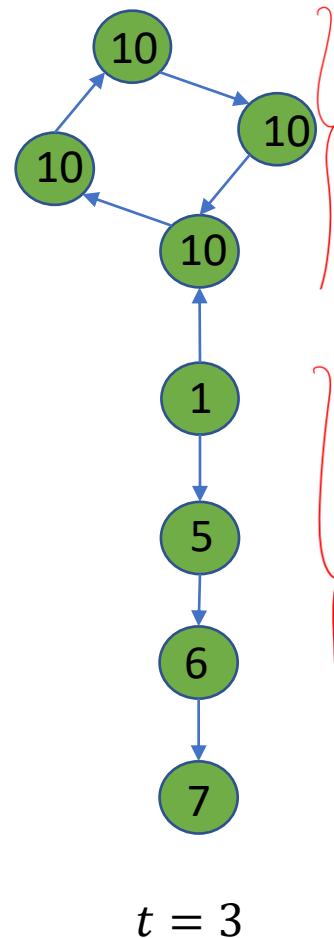
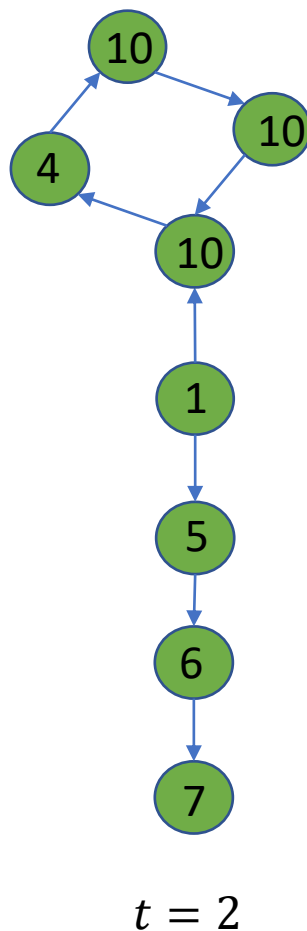
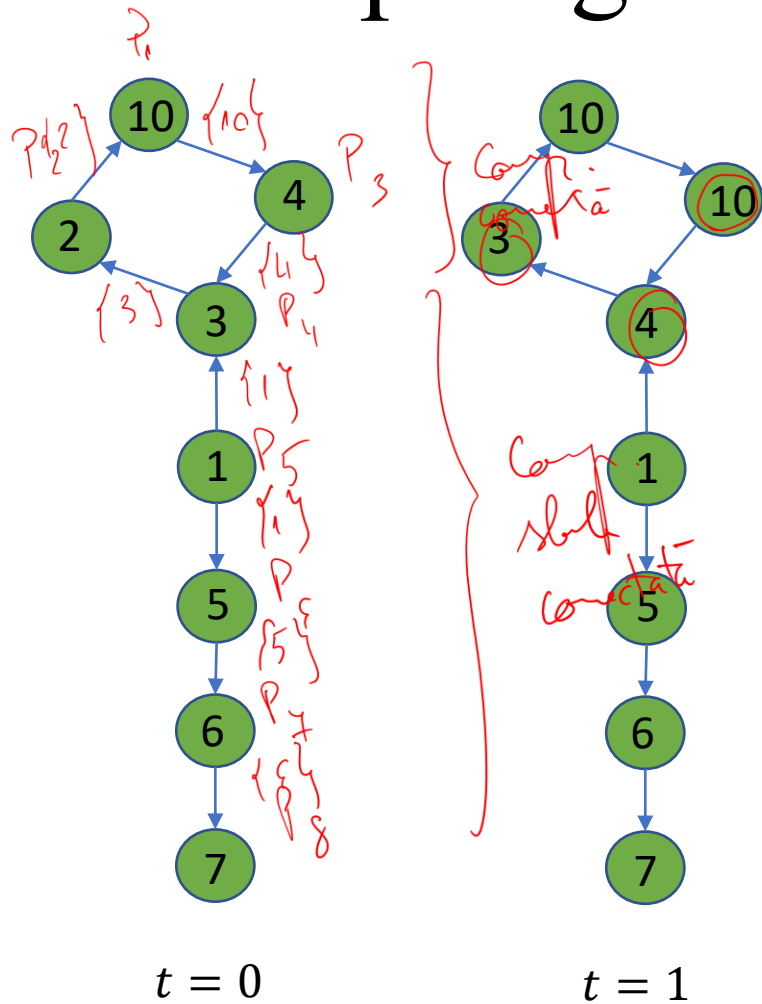
1. send(send_id, dreapta); *vein iese*
2. recv(rcv_id, stânga); *vein intrare*
3. **If** (rcv_id > id):
 1. send_id := rcv_id;
4. **ElseIf** (rcv_id == id): status = leader;
5. **ElseIf** (rcv_id < id): send_id := NULL;

Teorema [Lynch]. Algoritmul LCR rezolvă problema alegerii liderului.

Complexitate. Complexitatea timp este n iterații până la anunțarea unui lider, iar complexitatea mesaj este $O(n^2)$.

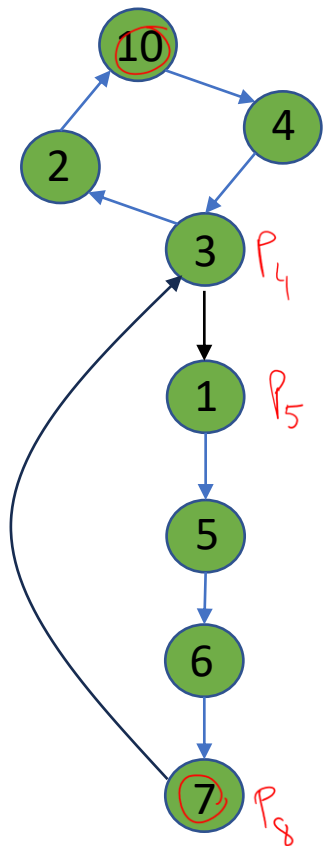


Alte topologii

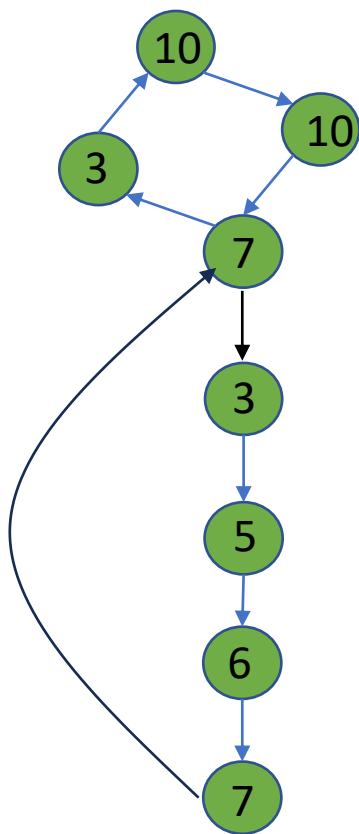


Componenta conexă
calculează corect index-ul
liderului.

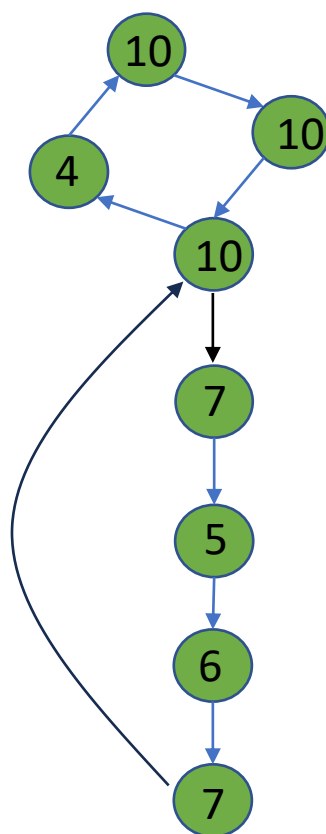
Alte topologii



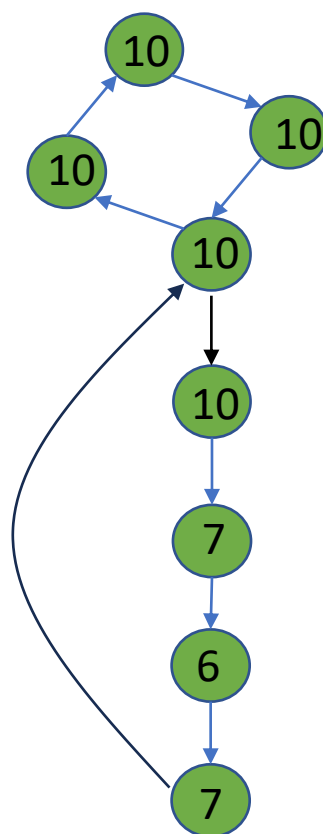
$t = 0$



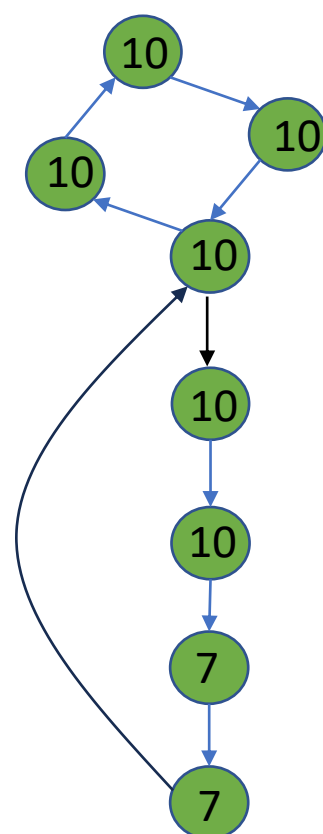
$t = 1$



$t = 2$

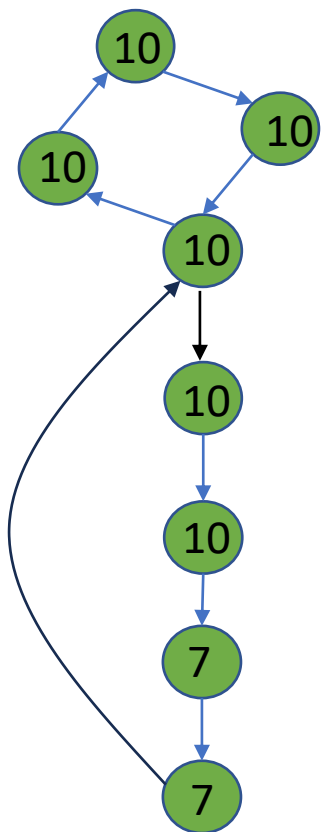


$t = 3$

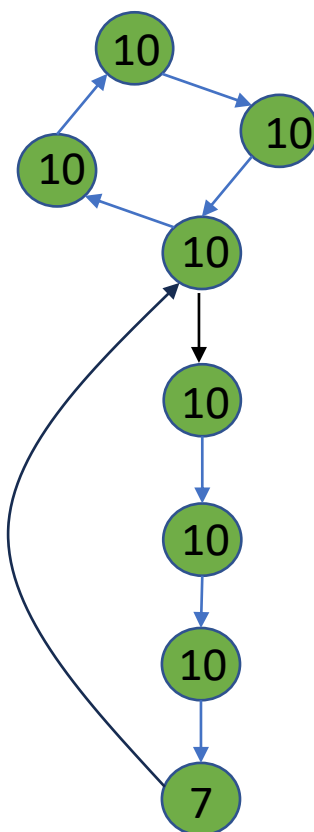


$t = 4$

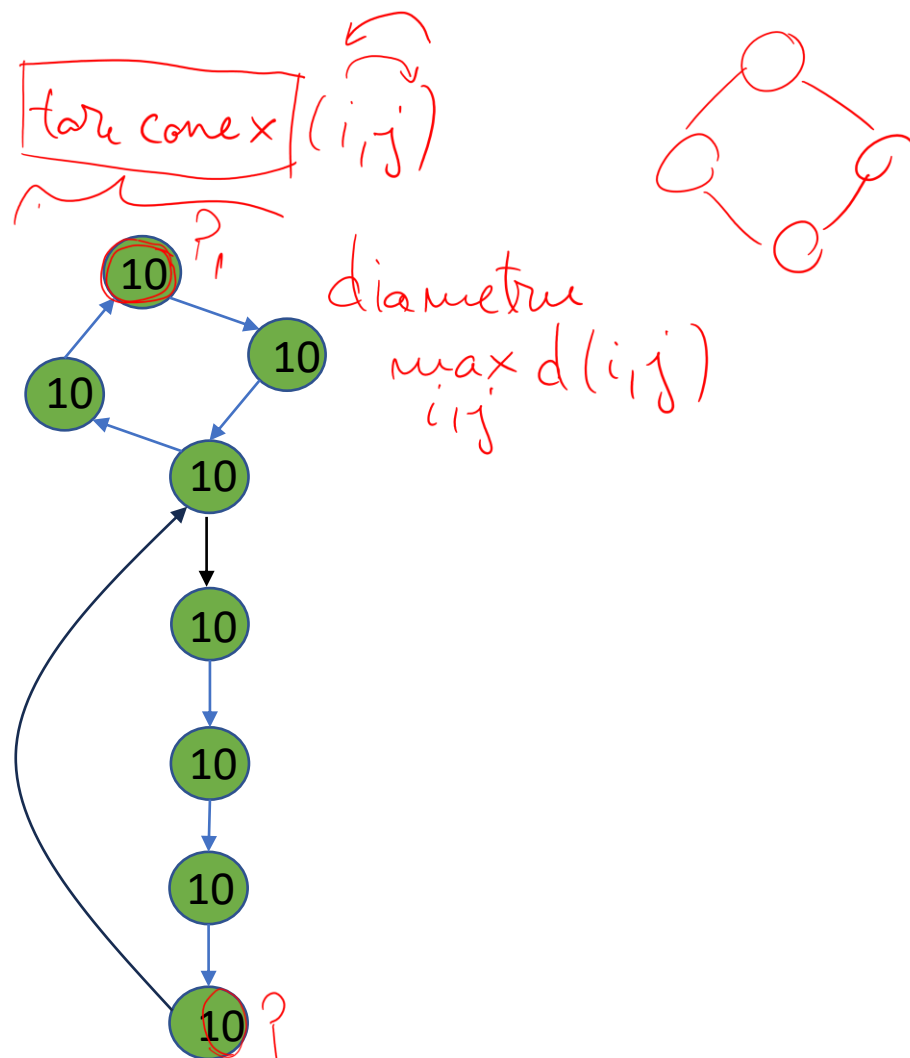
Alte topologii



$t = 4$



$t = 5$



$t = 6$

Algoritmul LCR pentru topologii generale

- Reconsiderăm problema AL, de data aceasta într-o topologie generală reprezentată printr-un graf directat tare conectat.
- Fiecare nod are un ID unic, ales dintr-un spațiu total ordonat.
- Un singur nod poate fi ales LIDER în rețea.
- Ideea unui algoritm simplu (extensie LCR):
 - Fiecare proces înregistrează ID-ul maxim cunoscut până în prezent
 - La fiecare iterație, nodurile propagă acest maxim pe toate muchiile de ieșire
 - După un număr de iterații egal cu diametrul grafului, dacă ID-ul maxim înregistrat coincide cu ID-ul local, nodul se va declara LIDER, altfel NON-LIDER.

Algoritmul Flooding (graf tare conectat)

Algoritm **Flooding_Gen**(max()):

M_i : - int id (token id)

- int max_id (var auxiliară), inițial id

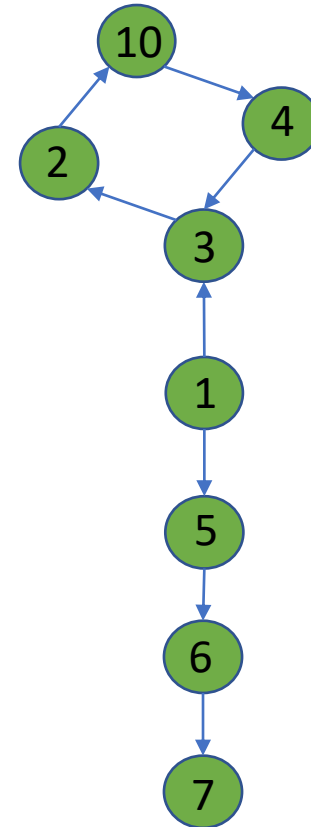
- $status \in \{lider, non-lider\}$, inițial non-lider

- int $rounds$, integer, inițial 0

- int $diam$ (estimare diametru graf)

Funcție transformare nod i ():

1. $t := t + 1$
2. Fie U mulțimea ID-urilor primite de la vecinii de intrare
3. $max_id := \max(\{max_id\} \cup U)$
4. If ($rounds == diam$): *approximativ*
 1. If ($max_id == id$): status = leader;
 2. Else: status = non-leader;
5. Else: send(max_id , vecini ieșire) *broadcast*



Algoritmul Flooding (graf tare conectat)

Algoritm **Flooding_Gen**(max()):

M_i : - int id (id propriu)

- int max_id (var auxiliară), inițial id

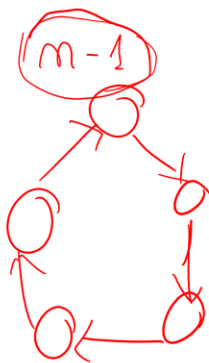
- $status \in \{lider, non-lider\}$, inițial non-lider

- int $rounds$, integer, inițial 0

- int $diam$ (diametru graf)

Funcție transformare nod i ():

1. $t := t + 1$
2. Fie U mulțimea ID-urilor primite de la vecinii de intrare
3. $max_id := \max(\{max_id\} \cup U)$
4. **If** ($rounds == diam$):
 1. **If** ($max_id == id$): $status = leader$;
 2. **Else**: $status = non-leader$;
5. **Else**: $send(max_id, \underline{vecini\ ieșire})$



Teorema [Lynch]. Presupunem graful topologiei tare conectat. În algoritmul Flooding_Gen, nodul cu indicele i_{max} devine *lider*, iar restul nodurilor *non-lider*, după cel mult $diam$ iterații.

Complexitate. Complexitatea timp este $diam$ iterații până la anunțarea unui lider, iar complexitatea mesaj este $diam \cdot |E|$. Prin $|E|$ înțelegem numărul de muchii directate din graf.

Remarci.

1. Flooding_Gen reprezintă o generalizarea algoritmului LCR la grafuri conectate;
2. LCR nu necesită informație globală;
3. Dacă graful = inel unidirecțional, atunci $diam \cdot |E| = (n - 1) \cdot n \approx n^2$;
4. Algoritmul funcționează cu o aproximare a constantei $diam$;

Algoritmul Flooding (alte aplicații)

- Difuzare în rețea (Broadcast)
- Arbori Breadth-First Search
- Consens (sincronizat)
- Estimări globale:
 - Diametru
 - Număr de noduri
 - Calcul distribuit